G. PULLAIAH COLLEGE OF ENGINEERING AND TECHNOLOGY
Autonomous · Affiliated to JNTUA, Anantapur · Kurnool

A40531 · DEEP LEARNING · IV B.Tech CSE · I Semester

UNIT I

Model Answers for the 10-Mark Question Bank

Derivations · formulas · diagrams · examination keywords

Linear Algebra, Probability and Numerical Computation

Q1 PCA procedure using covariance and eigen-decomposition

Q2 Eigen-decomposition versus SVD as routes to PCA

Q3 Overflow and underflow with numerical safeguards

Q4 Expectation, variance, covariance and the motivation for PCA

Q5 Gradient-descent derivation and the effect of the learning rate

Q6 Constrained optimisation with a norm constraint and regularisation

Q7 Least squares: normal equations and uniqueness

Q8 PCA before a neural network with correlated features

Every answer is written to the 10-mark pattern: definition, derivation, formula, diagram, comparison table and conclusion, followed by the keywords that examiners look for.

08
MODEL ANSWERS

57
NUMBERED FORMULAE

18
VECTOR DIAGRAMS

Contents

Unit I — Linear Algebra, Probability and Numerical Computation

Q1. PCA procedure 3

Q2. Eigen vs SVD 7

Q3. Overflow and underflow 9

Q4. Expectation, variance, covariance 12

Q5. Gradient descent 15

Q6. Constrained optimisation and regularisation 17

Q7. Least squares 20

Q8. PCA before a neural network 22

The unit at a glance

Q	Core idea	Result to memorise
1	Covariance → eigen-decomposition → projection	$C = W\Lambda W^\top$, $Z = XW_k$
2	SVD reaches the same subspace without forming C	$X = U\Sigma V^\top$, $\lambda_i = \sigma_i^2/(m-1)$
3	Never exponentiate large numbers; shift by the maximum	$\log \Sigma_j e^{z_j} = a + \log \Sigma_j e^{z_j - a}$
4	Moments summarise the cloud; PCA diagonalises C	$C = 1/(m-1)X^\top X$, $\mathrm{trace}(C) = \Sigma_i \lambda_i$
5	Steepest descent with a curvature-limited step	$\theta \leftarrow \theta - \eta \nabla J$, $\eta < 2/\lambda_{max}$
6	A norm constraint is a penalty: KKT ⇔ regulariser	$\min J$ s.t. $\|\theta\| \le c \Leftrightarrow \min J + \lambda\|\theta\|^2$
7	Least squares = orthogonal projection	$X^\top Xw = X^\top y$
8	PCA decorrelates and conditions the network's input	$\mathrm{Cov}(Z) = W_k^\top C W_k = \Lambda_k$

HOW TO USE THIS BOOKLET

Each answer follows the same skeleton, so you can revise the *pattern* once and then recall the content: **idea → derivation → formula → diagram → table → conclusion**.

Formula boxes carry equation numbers such as (3.2) — that is *Question 3, second equation*. Figure numbers follow the same rule, so (Fig. 5.1) belongs to Q5.

The **Golden line** is the one sentence to write at the end of an answer; the **Keywords** block is the list an examiner scans for. Aim to reproduce the boxed formulae exactly and to reproduce the diagrams approximately.

Q1 **Given a centered data matrix X, develop a complete PCA procedure using covariance / eigen-decomposition. Explain how the principal components are selected and how dimensionality is reduced.**

CO1 · L5 (Design) · 10 Marks

Aim and notation

Principal Component Analysis (PCA) is an **unsupervised linear transformation** that maps data of dimension n into a smaller number k of new, uncorrelated axes called **principal components**. Let

$$X \in \mathbb{R}^{m \times n}, \quad m = \text{number of samples}, n = \text{number of features} \tag{1.1}$$

where each row $x^{(i)}$ is one sample and each column is one feature. PCA answers a single optimisation question: *which direction w makes the projected data Xw spread out the most?* Maximum spread means maximum retained information.

Step 1 — Centre the data

The question already supplies a **centred** matrix, so the feature means are zero: $\bar{x}_j = 0$. If raw data were given, we must subtract the mean first:

$$X \leftarrow X - \mathbf{1}\mu^\top, \qquad \mu = \frac{1}{m}\sum_{i=1}^{m} x^{(i)} \tag{1.2}$$

Why centring is compulsory: PCA maximises variance. Variance is measured about the mean, and $X^\top X$ equals the scatter about the origin only when the mean is zero. With uncentred data the first component simply points towards the mean vector, which carries no information about spread.

Step 2 — Build the covariance matrix

$$C = \frac{1}{m-1} X^\top X \in \mathbb{R}^{n \times n} \tag{1.3}$$

The diagonal entry C_{jj} is the variance of feature j; the off-diagonal entry C_{jk} is the covariance between features j and k. Hence C encodes both spread and redundancy. The divisor $m-1$ gives the unbiased estimator; with large m the difference from m is negligible.

Properties of C: square $(n \times n)$, real, **symmetric** $(C^\top = C)$, and **positive semi-definite** $(u^\top C u = 1/(m-1)\|Xu\|^2 \geq 0$ for every $u)$. Symmetry guarantees real eigenvalues and mutually orthogonal eigenvectors; positive semi-definiteness guarantees $\lambda_i \geq 0$.

Step 3 — Eigen-decomposition of the covariance matrix

$$C = W \Lambda W^\top, \qquad W = [w_1, w_2, \ldots, w_n], \quad \Lambda = \text{diag}(\lambda_1, \ldots, \lambda_n) \tag{1.4}$$

Because C is symmetric, W is **orthonormal** ($W^\top W = I$, so $W^{-1} = W^\top$) and the λ_i are real and non-negative. The eigenvectors are the candidate principal directions and the eigenvalues are the variances along them.

Step 4 — Why the eigenvectors maximise variance (derivation)

For a unit direction w the variance of the projected data is the Rayleigh quotient:

$$\text{Var}(Xw) = \frac{1}{m-1} w^\top X^\top X w = w^\top C w \tag{1.5}$$

We therefore solve the **constrained maximisation**

$$\max_{w} w^\top C w \quad \text{subjectto} \quad w^\top w = 1 \tag{1.6}$$

Form the Lagrangian $L(w, \lambda) = w^\top C w - \lambda(w^\top w - 1)$ and set the gradient to zero:

$$\frac{\partial L}{\partial w} = 2Cw - 2\lambda w = 0 \Longrightarrow Cw = \lambda w \tag{1.7}$$

So the stationary directions are exactly the eigenvectors, and at such a point the achieved variance is

$$w^\top Cw = w^\top(\lambda w) = \lambda\,(w^\top w) = \lambda \tag{1.8}$$

Conclusion: the largest eigenvalue gives the maximum variance, so *eigenvalue = variance captured by that component*. The second component is found by repeating the maximisation subject to $w_2^\top w_1 = 0$ — hence successive principal components are orthogonal (uncorrelated).

Step 5 — Sort and select the top k components

Arrange $\lambda_1 \geq \lambda_2 \geq \ldots \geq \lambda_n \geq 0$ and reorder the columns of W identically. The importance of component i is measured by the **explained variance ratio**:

$$\mathrm{EVR}_i = \frac{\lambda_i}{\sum_{j=1}^{n}\lambda_j}, \qquad \mathrm{Cum}_k = \frac{\sum_{i=1}^{k}\lambda_i}{\sum_{i=1}^{n}\lambda_i} \tag{1.9}$$

Because $\Sigma_i \lambda_i = \mathrm{trace}(C)$ is the **total variance** of the data, Cum_k is the fraction of total information retained. The dimension k is chosen by any of the following standard rules:

- **Cumulative variance rule (most common):** smallest k with $\mathrm{Cum}_k \geq 0.95$ (or 0.99).
- **Scree plot / elbow rule:** plot λ_i against i and cut where the curve flattens.
- **Kaiser rule (standardised data only):** keep components with $\lambda_i > 1$, i.e. above average variance.
- **Task-driven rule:** choose k by cross-validation on the final classifier, not by variance alone.

The retained matrix is

$$W_k = [w_1, w_2, \ldots, w_k] \in \mathbb{R}^{n\times k}, \qquad k < n \tag{1.10}$$

Step 6 — Project the data (dimensionality reduction)

$$Z = XW_k \in \mathbb{R}^{m\times k}, \qquad z^{(i)} = W_k^\top x^{(i)} \tag{1.11}$$

Each sample of n features becomes a vector of k **principal component scores**. The dimension of the representation drops from n to k while the dominant variance is preserved. The transformation is a rotation followed by a drop of coordinates — no information in the retained subspace is destroyed.

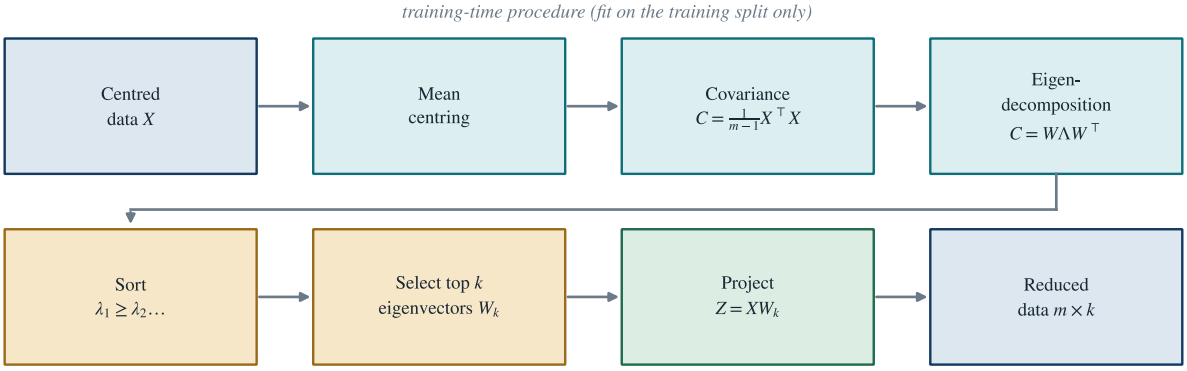


Fig. 1.1 Complete PCA chain: fitting the transform (top row) and the wrap-around continuation to the reduced representation (bottom row).

Step 7 — Properties of the reduced representation

$$\mathrm{Cov}(Z) = W_k^\top CW_k = \Lambda_k = \mathrm{diag}(\lambda_1, \ldots, \lambda_k) \tag{1.12}$$

The covariance of the scores is **diagonal**, so the new features are uncorrelated: PCA simultaneously performs dimensionality reduction **and** decorrelation. Total variance is unchanged by the rotation, $\Sigma_{i=1}^{n}\lambda_i = \text{trace}(C)$, and the first k components carry $\Sigma_{i=1}^{k}\lambda_i$ of it.

Step 8 — Reconstruction and the error it causes

Discarded components can be dropped, but they can also be put back approximately:

$$\hat{x} = W_k z = W_k W_k^{\top} x \approx x, \qquad \|X - \hat{X}\|_F^2 = \sum_{i=k+1}^{n} \lambda_i \tag{1.13}$$

This is the **Eckart–Young theorem**: among all rank-k approximations, the PCA truncation has the smallest squared reconstruction error, and that error is exactly the total variance of the discarded components. It justifies dropping the smallest eigenvalues rather than any other directions.

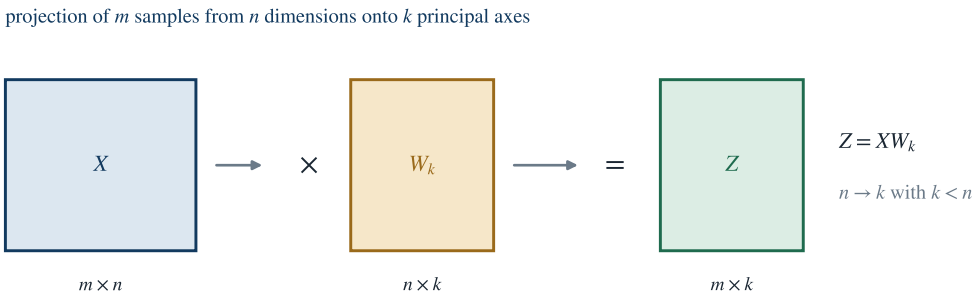


Fig. 1.2 Dimensions of the three matrices involved in the projection step.

Worked numerical example

Take four centred 2-D samples $x^{(1)} = (2, 1)$, $x^{(2)} = (1, 2)$, $x^{(3)} = (-1, -2)$, $x^{(4)} = (-2, -1)$, so $m = 4$, $n = 2$ and the mean is already zero.

Step	Computation	Result
Covariance	$C = X^{\top}X/(m-1)$	diag = variances, off-diag = covariances
Eigenvalues	$\det(C - \lambda I) = 0$	$\lambda_1 = 6, \lambda_2 = 0.667$
Eigenvectors	$Cw = \lambda w$	$w_1 = (0.707, 0.707)^{\top}$, $w_2 = (-0.707, 0.707)^{\top}$
Variance share	$\lambda_i/\Sigma\lambda$	90% and 10%
Projection	$Z = XW_1$	$z_1 = (2.12, 2.12, -2.12, -2.12)$

With $\text{Cum}_1 = 0.90$, the one-dimensional projection keeps 90% of the variance: four samples that needed two numbers each are now described by one number each, and the reconstruction error is $\lambda_2 = 0.667$, exactly the dropped variance.

$$C = \frac{1}{m-1}X^{\top}X = \begin{bmatrix} 3.33 & 2.67 \\ 2.67 & 3.33 \end{bmatrix}, \quad Cw = \lambda w \Rightarrow \begin{bmatrix} 6.00 \\ 0.667 \end{bmatrix} \tag{1.14}$$

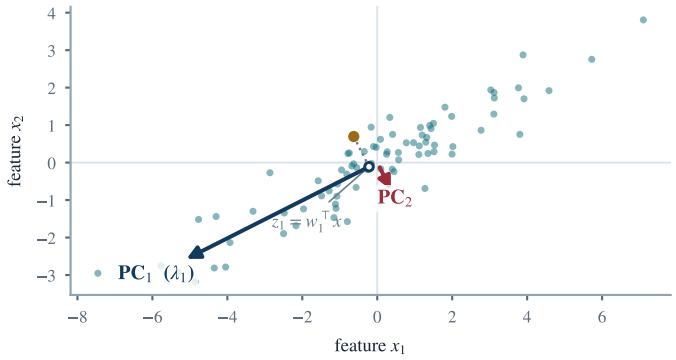


Fig. 1.3 Geometry of the first step: the first principal axis w_1 follows the direction of largest spread, and every point is compressed onto it by $z_1 = w_1^\top x$.

Algorithm (write this in the exam)

1. Centre the data: $X \leftarrow X - \mathbf{1}\mu^\top$.
2. Compute the covariance matrix $C = 1/(m-1)X^\top X$.
3. Eigen-decompose $C = W\Lambda W^\top$.
4. Sort eigenvalues in descending order and reorder W.
5. Choose k from the cumulative explained variance (e.g. $\geq 95\%$).
6. Form W_k from the first k eigenvectors and compute $Z = XW_k$.
7. Optionally reconstruct $\hat{X} = ZW_k^\top$ and report the residual $\Sigma_{i>k}\lambda_i$.

Limitations

- PCA is **linear**; curved manifolds need kernel PCA, autoencoders or t-SNE.
- Components are chosen by **variance, not by relevance** to the target y — a low-variance but highly predictive direction may be discarded.
- PCA is **scale sensitive**: features must be standardised first, otherwise the largest-unit feature dominates the first component.
- Components are linear combinations of all inputs, so interpretability is reduced.
- The eigen route costs $O(n^3)$ (or $O(mn^2)$), which is expensive when n is very large.

EXAMINER'S CHECKLIST

Always state (i) centring, (ii) the covariance formula with $m-1$, (iii) $C = W\Lambda W^\top$, (iv) *eigenvalue = variance*, (v) $Z = XW_k$ with dimensions, and (vi) the cumulative-variance rule for k. Those six items carry most of the ten marks.

GOLDEN LINE

Largest eigenvalue = maximum variance = PC1. Projection $Z = XW_k$ turns n features into k uncorrelated features with minimum loss.

KEYWORDS TO WRITE IN THE EXAM

Principal component · covariance matrix · eigen-decomposition · eigenvalue = variance · orthonormal eigenvectors · explained variance ratio · cumulative variance · $Z = XW_k$ · decorrelation · Eckart–Young · dimensionality reduction

Q2 **Compare eigen-decomposition and SVD as routes to PCA. For a rectangular data matrix, analyse which decomposition is more directly applicable and justify your choice.**

CO1 · L4 (Analyse) · 10 Marks

The two routes

PCA needs only one thing: the orthonormal directions of maximum variance. Those directions can be obtained in two mathematically different ways — from the **square covariance matrix** (eigen-decomposition) or from the **data matrix itself** (singular value decomposition). Both end at the same subspace; they differ in what they factorise, in numerical behaviour and in cost.

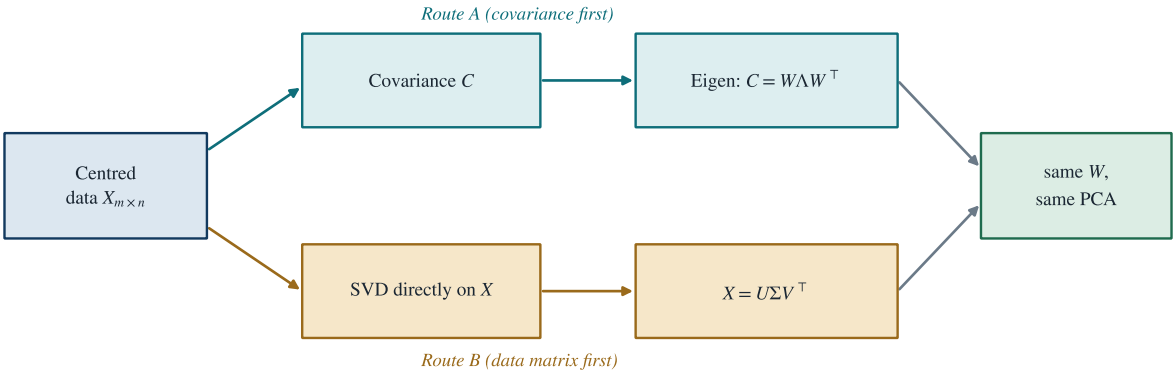


Fig. 2.1 Route A builds the covariance matrix first; Route B factorises the rectangular data matrix directly and reaches the same principal directions.

Route A — eigen-decomposition of the covariance matrix

$$C = \frac{1}{m-1} X^\top X \in \mathbb{R}^{n \times n}, \qquad C = W \Lambda W^\top \tag{2.1}$$

C is square and symmetric, so its eigenvectors are orthonormal and its eigenvalues are real and non-negative. Sorting gives $\lambda_1 \geq \lambda_2 \geq \ldots \geq \lambda_n \geq 0$; the corresponding eigenvectors w_i are the principal directions and the scores are $Z = XW_k$.

Route B — singular value decomposition of the data matrix

For **any** real matrix, square or rectangular, the SVD exists:

$$X = U \Sigma V^\top, \qquad U^\top U = I, V^\top V = I \tag{2.2}$$

Here $U \in \mathbb{R}^{m \times r}$ holds the left singular vectors (normalised scores), $\Sigma = \mathrm{diag}(\sigma_1, \ldots, \sigma_r)$ holds the singular values with $\sigma_1 \geq \sigma_2 \geq \ldots \geq 0$, and $V \in \mathbb{R}^{n \times r}$ holds the right singular vectors, which **are the principal directions** ($W = V$). The reduced representation is $Z = XV_k = U_k \Sigma_k$.

Derivation of the equivalence

Substituting the SVD into the covariance matrix gives

$$X^\top X = (U \Sigma V^\top)^\top (U \Sigma V^\top) = V \Sigma^\top U^\top U \Sigma V^\top = V \Sigma^2 V^\top \tag{2.3}$$

where $U^\top U = I$ removes the middle factor. Comparing with $C = W \Lambda W^\top$ term by term:

$$W = V \qquad \text{and} \qquad \Lambda = \frac{\Sigma^2}{m-1} \tag{2.4}$$

Hence the **eigenvalues are the scaled squares of the singular values**, the eigenvectors are the right singular vectors, and the ordering is identical because squaring preserves order. This is the exact statement that both routes give the same PCA.

$$\lambda_i = \frac{\sigma_i^2}{m-1}, \qquad \mathrm{EVR}_i = \frac{\sigma_i^2}{\sum_j \sigma_j^2} \tag{2.5}$$

Because the factor $1/(m-1)$ cancels in the ratio, the explained-variance percentages can be read straight from the squared singular values — the covariance matrix need never be formed.

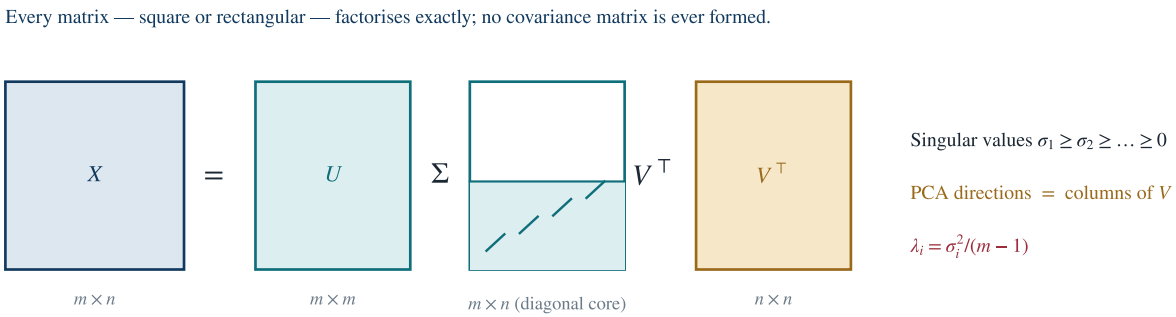


Fig. 2.2 Shapes in the full SVD of a rectangular matrix; only the first r columns matter, giving the economy (thin) SVD.

Comparison

Criterion	Eigen-decomposition	Singular value decomposition
Object factorised	$C = 1/(m-1)X^\top X$ (square, $n \times n$)	X itself ($m \times n$, any shape)
Applicability	Only square matrices — the covariance must be built first	Every real matrix, square or rectangular
Output	W (eigenvectors), Λ (variances)	U, Σ, V; $W = V$, $\lambda_i = \sigma_i^2/(m-1)$
Cost	$O(mn^2)$ to form C + $O(n^3)$ to factorise	$O(mn\min(m,n))$, no $n \times n$ matrix formed
Numerical accuracy	Squaring doubles the condition number: $\kappa(X^\top X) = \kappa(X)^2$	Works with $\kappa(X)$ — no squaring, hence more accurate
Catastrophic case	Fails when $m < n$: C is singular, rank $\le m$	Still exact; returns $\min(m,n)$ nonzero singular values
Sign convention	Signs of w_i arbitrary	Signs of u_i, v_i arbitrary (same ambiguity)
Practical use	Small n, or when C is already available	Default choice: NumPy *svd*, scikit-learn PCA, large or wide data

Why SVD is the more direct route for a rectangular matrix

A rectangular $X \in \mathbb{R}^{m \times n}$ has no eigenvectors — "eigen" is defined only for square operators. The eigen route therefore inserts an artificial intermediate step: it first squares the problem into $X^\top X$, factorises that, and then has to undo the scaling. SVD needs no such step; it factorises X exactly as given. Four concrete consequences follow.

1. **It always exists.** The SVD is defined for every matrix and always has $\min(m,n)$ terms, whereas $X^\top X$ is singular whenever $m < n$ (wide data, e.g. 100 samples of 10{,}000 genes) and half of the eigen-spectrum is then numerically meaningless zero.
2. **It is numerically better conditioned.** Forming $X^\top X$ squares the condition number, so directions with small singular values can disappear into round-off. SVD works on X directly and retains them.
3. **It is cheaper.** Forming C costs $O(mn^2)$ and factorising it $O(n^3)$; a thin SVD costs $O(mn\min(m,n))$. For wide matrices the difference is decisive.
4. **It gives the scores for free.** $Z = U_k\Sigma_k$ comes out of the same factorisation, and the discarded energy $\Sigma_{i>k}\sigma_i^2$ is read directly from Σ.

The eigen route keeps two advantages: it is easy to explain in an exam, and when the covariance matrix is already given (as in Question 1) it is the natural object to work on. It is also the route that reveals *why* PCA maximises variance, through $Cw = \lambda w$.

When to prefer which

- **Use SVD** when the input is a raw rectangular data matrix, when n is large, when $m < n$, or when numerical accuracy matters — this is the practical default.
- **Use eigen-decomposition** when the covariance matrix already exists, when n is small, or when the theory ($Cw = \lambda w$, Rayleigh quotient) must be demonstrated.

COMMON MISTAKES THAT LOSE MARKS

(i) Saying "SVD and eigen give different PCs" — they give the *same* directions. (ii) Forgetting the factor $m - 1$ when converting σ_i^2 into λ_i. (iii) Claiming eigenvectors exist for a rectangular matrix. (iv) Ignoring that both methods leave the **sign** of each component undetermined, so a comparison of components can look different after a sign flip.

GOLDEN LINE

Eigen route: covariance first, then factorise. SVD route: factorise the data directly. Right singular vectors of X are the eigenvectors of $X^\top X$ — for a rectangular matrix, SVD is the more direct and numerically safer route.

KEYWORDS TO WRITE IN THE EXAM

Eigen-decomposition · singular value decomposition · $X = U\Sigma V^\top$ · $X^\top X = V\Sigma^2 V^\top$ · $\lambda_i = \sigma_i^2/(m-1)$ · rectangular matrix · condition number · economy SVD · sign ambiguity

Q3 A deep-learning implementation produces unstable numerical values because exponentials become extremely large or small. Analyse overflow and underflow and propose mathematically sound numerical safeguards.

CO1 · L5 (Design) · 10 Marks

Where the problem comes from

Deep learning is dominated by two operations: **exponentials** (softmax, sigmoid, Gaussian likelihood) and **long products** (layer compositions, likelihoods over many samples). A computer stores real numbers in floating point, IEEE-754 float32, which has a finite window of representable magnitudes. Outside that window the hardware cannot represent the true value and the result is not an error — it is silently replaced by 0 or $\pm\infty$.

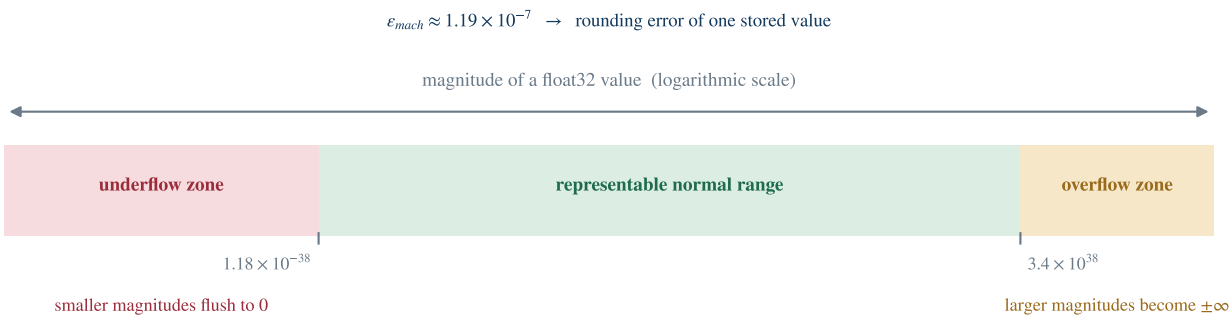


Fig. 3.1 Representable magnitudes of a float32 value: the underflow zone, the normal range and the overflow zone.

Key constants for float32: largest finite value $\approx 3.4 \times 10^{38}$, smallest normal positive $\approx 1.18 \times 10^{-38}$, machine epsilon $\approx 1.19 \times 10^{-7}$.

- **Overflow** occurs when a result exceeds the largest representable magnitude: $x > 3.4 \times 10^{38}$ becomes $+\infty$. Example: e^{1000} in a softmax, or a weight norm that explodes during training.
- **Underflow** occurs when a non-zero result becomes smaller than the smallest representable magnitude: $0 < |x| < 1.18 \times 10^{-38}$ is flushed to 0. Example: $\Pi_t p(y_t \mid y_{<t})$ over a 40-token sequence, or e^{-x} for large x.

- **Not-a-number (NaN)** is the secondary damage: $\infty - \infty$, $0 \cdot \infty$, ∞/∞, $0/0$ all produce NaN, and a single NaN propagates through every later gradient.

Why exponentials make this inevitable

For softmax with logits $z_1, \ldots, z_K$:

$$p_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)} \tag{3.1}$$

If any z_i is large, $\exp(z_i)$ overflows to ∞ and the ratio becomes $\infty/\infty =$ NaN. If all z_i are very negative, every $\exp(z_i)$ underflows to 0 and the ratio becomes $0/0 =$ NaN. The same failure occurs in the sigmoid $\sigma(x) = \frac{1}{1+e^{-x}}$, where e^{-x} overflows for very negative x; and in the Gaussian log-likelihood.

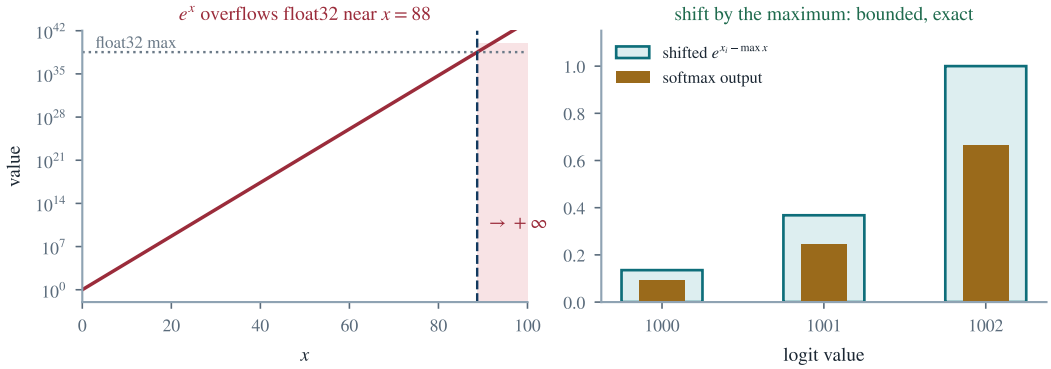


Fig. 3.2 Left: the exponential leaves the float32 range near $x = 88$. Right: shifting every logit by the maximum keeps all exponentials ≤ 1, so the softmax is computed exactly.

Safeguard 1 — the max-shift (log-sum-exp) trick

The softmax is **shift invariant**: multiplying numerator and denominator by e^{-a} changes nothing.

$$\frac{e^{z_i}}{\sum_j e^{z_j}} = \frac{e^{z_i - a}}{\sum_j e^{z_j - a}} \quad \text{for every } a \tag{3.2}$$

Choosing $a = \max_j z_j$ guarantees $z_i - a \leq 0$, so every exponential lies in $(0, 1]$: **overflow is impossible** and at least one term equals 1, so **total underflow is impossible**. Correctness is unchanged, only conditioning is improved.

$$\text{softmax}(z)_i = \frac{e^{z_i - \max_j z_j}}{\sum_j e^{z_j - \max_j z_j}}, \qquad \log \sum_j e^{z_j} = a + \log \sum_j e^{z_j - a} \tag{3.3}$$

The same identity gives the stable log-sum-exp, the workhorse for log-likelihoods and mixture models. In practice one never computes softmax and then a logarithm; one calls a single fused *log_softmax* primitive.

Safeguard 2 — work in the log domain

Products of probabilities become sums of log-probabilities, and sums do not overflow:

$$\log \prod_{t=1}^{T} p_t = \sum_{t=1}^{T} \log p_t \tag{3.4}$$

A likelihood of 10^{-300} underflows, but its log-likelihood -690.8 is an ordinary float. All objectives are therefore written as **negative log-likelihood**, and probabilities are exponentiated only at the final reporting step. Use *log1p(x)* instead of *log(1+x)* and *expm1(x)* instead of *exp(x)-1*, both of which keep full precision for small x.

Safeguard 3 — clip the arguments of exponentials and the gradients

$$\sigma(x) = \frac{1}{1+e^{-x}} (x \geq 0) \qquad \sigma(x) = \frac{e^{x}}{1+e^{x}} (x < 0) \tag{3.5}$$

In code: for $x \geq 0$ use $\sigma(x) = 1/(1 + e^{-x})$ (the exponential underflows harmlessly to 0, giving $\sigma = 1$); for $x < 0$ use $\sigma(x) = e^{x}/(1 + e^{x})$ (again a small, safe exponential). For the log-sigmoid use $-\text{softplus}(-x)$ with $\text{softplus}(u) = \log(1 + e^{u})$ computed stably, never $\log(\sigma(x))$ directly.

Gradient clipping caps the update so that a single explosive batch cannot destroy the weights:

$$g \leftarrow g \cdot \min\left(1, \frac{c}{\|g\|_2}\right) \tag{3.6}$$

This keeps the **direction** of the gradient and only rescales its magnitude, so learning continues instead of becoming NaN.

Safeguard 4 — condition the inputs and the parameters

- **Standardise inputs** ($x \leftarrow (x - \mu)/\sigma$) so that no layer sees values of magnitude 10^{6}.
- **Normalise activations** with batch/layer normalisation, which keeps pre-activations in a range where exponentials are safe.
- **Stable initialisation** (Xavier/He) keeps variances near 1 across layers, preventing products of matrices from exploding or vanishing.
- **Weight decay / regularisation** bounds $\|w\|$ so inner products $w^{\top} x$ cannot grow without limit.
- **Mixed precision with loss scaling:** compute in float16 but keep a master copy of weights in float32, and multiply the loss by a large scale factor so small gradients stay representable.

Safeguard 5 — avoid the numerically fragile formulations

Fragile expression	Why it fails	Numerically sound replacement
$\log(\text{softmax}(z))$	softmax underflows, $\log 0 = -\infty$	*log_softmax* fused with NLL
$w = (X^{\top}X)^{-1}X^{\top}y$	squaring doubles κ; singular $X^{\top}X$	QR or SVD: $w = V\Sigma^{-1}U^{\top}y$
$\sigma(x)$ then $\log \sigma(x)$	intermediate underflows to 0	$-\text{softplus}(-x)$
$\det(C)$ for log-likelihood	overflows/underflows for large n	Cholesky: $\log\det C = 2\Sigma_i \log L_{ii}$
$\Pi_i p_i$	underflows for long sequences	$\Sigma_i \log p_i$
$\exp(x) - \exp(y)$	cancellation for close values	stable form or *expm1*

Diagnostic procedure

1. Insert *assert torch.isfinite(loss).all()* after the loss; the first non-finite value localises the fault.
2. Check the loss at step 1: *inf* points to a softmax/likelihood, NaN points to a $0/0$ or $\infty - \infty$.
3. Log the maximum absolute value of each activation, gradient and weight. A growth of orders of magnitude per epoch is the signature of overflow.
4. Reduce the learning rate and enable gradient clipping to confirm that the cause is dynamics rather than a formula.
5. Once stable, re-introduce mixed precision with loss scaling.

HOW TO PHRASE THE ANSWER

Structure it as **cause** → **symptom** → **remedy**: overflow gives $\pm\infty$, underflow gives 0, both give NaN. Then give one mathematically justified safeguard per case (max-shift, log-domain, clipping) instead of merely saying "use float64".

GOLDEN LINE

Never exponentiate a large number and never multiply many small numbers. Subtract the maximum before exponentiating, add logarithms instead of multiplying probabilities, and clip the gradient norm.

KEYWORDS TO WRITE IN THE EXAM
Overflow · underflow · float32 range 3.4×10^{38} · machine epsilon · NaN · log-sum-exp trick · max-shift softmax · log-domain · log1p/expm1 · gradient clipping · stable sigmoid · log_softmax · loss scaling

Q4 Analyse how expectation, variance and covariance characterise a multivariate training dataset. Explain how the covariance structure motivates PCA.

CO1 · L4 (Analyse) · 10 Marks

Why these three quantities matter

A deep network sees only a cloud of m points in $\mathbb{R}^n$. Three statistics summarise that cloud completely for linear methods: **expectation** says where the cloud is centred, **variance** says how widely each coordinate spreads, and **covariance** says whether coordinates move together. Together they form the covariance matrix, which is the object PCA diagonalises.

Expectation

For a random variable X with density $p(x)$ and for a finite sample:

$$\mathbb{E}[X] = \int x\, p(x)\, dx \quad \approx \quad \hat{\mu} = \frac{1}{m}\sum_{i=1}^{m} x^{(i)} \tag{4.1}$$

For a vector $x \in \mathbb{R}^n$ the expectation is taken element-wise, $\mathbb{E}[x] = (\mathbb{E}[x_1], \ldots, \mathbb{E}[x_n])^\top$, and it is the **centre of mass** of the data. It is a linear operator:

$$\mathbb{E}[aX + bY] = a\,\mathbb{E}[X] + b\,\mathbb{E}[Y] \tag{4.2}$$

Variance

$$\mathrm{Var}(X) = \mathbb{E}\big[(X - \mathbb{E}[X])^2\big] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2 \tag{4.3}$$

Variance is the **second central moment**; it measures average squared deviation from the mean and therefore the spread of one coordinate. Its square root σ is in the same units as the data, which is why standardisation divides by σ. The unbiased sample estimator uses $m - 1$:

$$\hat{\sigma}^2 = \frac{1}{m-1}\sum_{i=1}^{m} (x_i - \hat{\mu})^2 \tag{4.4}$$

For an affine transformation, $\mathrm{Var}(aX + b) = a^2\mathrm{Var}(X)$ — shifts do not change spread, and scaling scales variance by the square.

Covariance

$$\mathrm{Cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}X)(Y - \mathbb{E}Y)] = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y] \tag{4.5}$$

Covariance measures **joint** variation. Its sign is informative:

- $\mathrm{Cov} > 0$: both variables grow together (redundant information).
- $\mathrm{Cov} < 0$: one grows while the other falls (also redundant).
- $\mathrm{Cov} = 0$: no *linear* relationship (uncorrelated).

The correlation coefficient removes the units, so it is the interpretable scale-free version:

$$\rho_{XY} = \frac{\mathrm{Cov}(X, Y)}{\sigma_X \sigma_Y} \in [-1, 1] \tag{4.6}$$

The covariance matrix

Collecting all pairwise covariances of a centred data matrix into one array gives the **covariance (scatter) matrix**:

$$C = \frac{1}{m-1} X^\top X \in \mathbb{R}^{n \times n}, \qquad C_{jk} = \mathrm{Cov}(x_j, x_k) \tag{4.7}$$

- The **diagonal** $C_{jj} = \mathrm{Var}(x_j)$ gives the variance of each feature.
- The **off-diagonal** C_{jk} gives the redundancy between features j and k.
- C is symmetric and positive semi-definite, and its trace is the total variance: $\mathrm{trace}(C) = \Sigma_j \mathrm{Var}(x_j)$.

Under an affine map $z = Ax + b$ the covariance transforms as

$$\mathrm{Cov}(Az) = A\,\mathrm{Cov}(z)\,A^\top \tag{4.8}$$

This single identity is the engine of PCA: choosing $A = W^\top$ with W orthonormal gives $\mathrm{Cov}(W^\top x) = W^\top C W$, so PCA is the search for an A that makes the result diagonal.

How the covariance structure motivates PCA

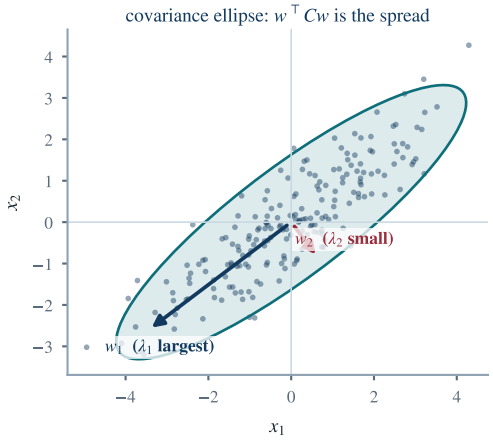


Fig. 4.1 The covariance ellipse of a 2-D cloud. Its axes are the eigenvectors of C, and their lengths are $\sqrt{\lambda_i}$; w_1 is the direction of maximum variance.

1. **Correlated features mean redundant coordinates.** If C has large off-diagonals, several columns of X carry nearly the same information. The data therefore occupies a thin, tilted region of $\mathbb{R}^n$, not a full box. Describing that region with its own axes needs fewer numbers than n.
2. **Variance is information.** A direction with near-zero variance is almost constant across samples and contributes almost nothing to distinguishing them. Dropping it loses almost nothing.
3. **The best axes are the eigen-directions of C.** For a unit direction w, the variance of the projection is the Rayleigh quotient $w^\top Cw$; maximising it gives $Cw = \lambda w$, i.e. eigenvectors, and the achieved variance is the eigenvalue.
4. **Diagonalisation removes redundancy.** With W the eigenvector matrix, $\mathrm{Cov}(W^\top x) = W^\top C W = \Lambda$ is diagonal: the new coordinates are uncorrelated. PCA is literally a change of basis into the eigenbasis of C.
5. **Total variance is conserved but redistributed.** Since $\mathrm{trace}(C) = \Sigma_i \lambda_i$, the rotation creates no new information; it concentrates the same total variance into the first few coordinates, which is exactly what makes truncation possible.

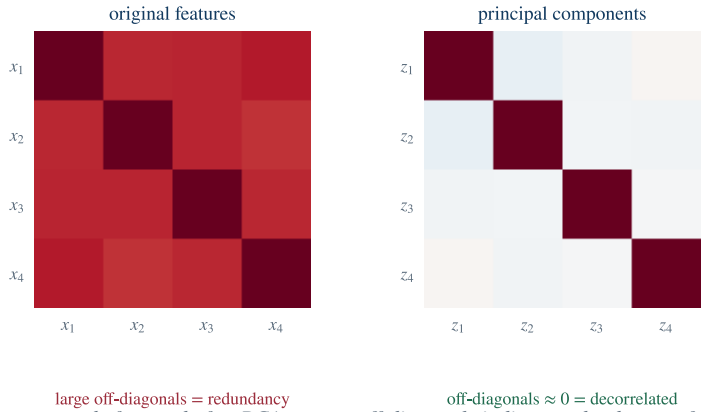


Fig. 4.2 Covariance (correlation) structure before and after PCA: strong off-diagonals indicate redundancy; after rotation the matrix is essentially diagonal.

Worked illustration

For the four-sample example of Question 1, C has diagonal 3.33 and off-diagonal 2.67. The diagonal entries say each feature has variance 3.33; the off-diagonal 2.67 is the same order of magnitude, so the two features are strongly redundant ($\rho = 0.8$). The eigenvalues 6 and 0.667 show why: one direction concentrates 90% of the total variance 6.67, the other only 10%. Averaging the two features along $w_1 = (1, 1)/\sqrt{2}$ *is* the first principal component.

Statistic	Formula	What it tells about the dataset	Role in PCA
Expectation	$\mu = 1/m \Sigma x^{(i)}$	location of the cloud	must be removed (centring)
Variance	$\mathrm{Var}(x_j) = \mathbb{E}[(x_j - \mu_j)^2]$	spread of one feature	becomes an eigenvalue λ_i
Covariance	$\mathrm{Cov}(x_j, x_k)$	linear redundancy between features	off-diagonal that PCA drives to zero
Covariance matrix	$C = 1/(m-1) X^\top X$	complete second-order description	factorised as $W \Lambda W^\top$
Trace	$\mathrm{trace}(C) = \Sigma_j \mathrm{Var}(x_j) = \Sigma_i \lambda_i$	total variance	denominator of the explained-variance ratio

Practical cautions

- Covariance captures only **linear** dependence: $\mathrm{Cov} = 0$ does not imply independence (e.g. $Y = X^2$).
- It is **not scale invariant**; features in different units must be standardised, which replaces C by the correlation matrix R.
- A single outlier can dominate C because deviations are squared; robust variants (trimmed or Huber covariance) exist.
- The sample covariance needs $m > n$ to be full rank; otherwise C is singular, which is another reason to prefer SVD.

LINK TO THE REST OF UNIT I

Covariance is the bridge between probability and linear algebra: Question 1 maximises $w^\top C w$, Question 2 factorises $X^\top X$, Question 7 solves $X^\top X w = X^\top y$, and Question 8 uses a diagonal covariance to speed up training. Writing that link explicitly earns analytical marks.

GOLDEN LINE

Expectation locates the cloud, variance measures its spread along one axis, and covariance measures how two axes move together. PCA rotates the axes so that the covariance matrix becomes diagonal, concentrating total variance into the fewest coordinates.

KEYWORDS TO WRITE IN THE EXAM

Expectation · variance · standard deviation · covariance · correlation · covariance matrix · positive semi-definite · trace = total variance · Rayleigh quotient · diagonalisation · decorrelation

Q5 **Derive the gradient-descent update for a general differentiable objective $J(\theta)$. Analyse how an excessively large and an excessively small learning rate affect convergence.**

CO1 · L4 (Analyse) · 10 Marks

The optimisation problem

Training a network means solving

$$\theta^* = \arg\min_{\theta \in \mathbb{R}^p} J(\theta) \tag{5.1}$$

where θ collects all weights and biases (p parameters). For a deep network J is non-convex and has no closed-form minimiser, so we iterate. Gradient descent is the simplest first-order iterative method: it uses only ∇J, never the Hessian.

First-order (Taylor) derivation

Expand the objective about the current point θ_k:

$$J(\theta_k + \Delta) \approx J(\theta_k) + \nabla J(\theta_k)^\top \Delta \tag{5.2}$$

We must choose a step Δ that decreases J, i.e. $\nabla J^\top \Delta < 0$. Among steps of fixed length $\|\Delta\| = \varepsilon$, the linear term is minimised when Δ is **antiparallel** to the gradient (Cauchy–Schwarz: $\nabla J^\top \Delta \geq -\|\nabla J\|\,\|\Delta\|$, with equality when $\Delta \propto -\nabla J$). Hence the direction of steepest descent is $-\nabla J$, and the update is

$$\theta_{k+1} = \theta_k - \eta \nabla J(\theta_k), \qquad \eta > 0 \tag{5.3}$$

This is gradient descent; η is the **learning rate** (step size). Written component-wise:

$$\theta_{k+1}^{(j)} = \theta_k^{(j)} - \eta \frac{\partial J}{\partial \theta^{(j)}}(\theta_k) \tag{5.4}$$

Working rule: if $\partial J/\partial \theta > 0$, increasing θ increases J, so θ moves down (subtract); if the derivative is negative, θ moves up. The minus sign enforces both cases.

Second-order justification and the bound on η

Assume J is L-smooth, i.e. $\|\nabla J(u) - \nabla J(v)\| \leq L\|u - v\|$, where $L = \lambda_{max}(\nabla^2 J)$ near a minimum. Then

$$J(\theta - \eta \nabla J) \leq J(\theta) - \eta\left(1 - \frac{\eta L}{2}\right)\|\nabla J\|^2 \tag{5.5}$$

The step **decreases** the objective whenever $0 < \eta < 2/L$. The curvature term $(1 - \eta L/2)$ is largest at $\eta = 1/L$, so $1/L$ is the theoretical optimal constant step. This is why the useful learning rate is set by the *curvature* of the loss, not by guessing.

Convergence on a quadratic

For the quadratic $J(\theta) = 1/2\theta^\top H\theta$ with H symmetric positive definite, the update is linear and can be solved exactly. Let $e_k = \theta_k - \theta^*$; then

$$e_{k+1} = e_k - \eta H e_k = (I - \eta H)e_k \Longrightarrow e_k = (I - \eta H)^k e_0 \tag{5.6}$$

In the eigenbasis of H this decouples into $e_i^{(k)} = (1 - \eta\lambda_i)^k e_i^{(0)}$. Convergence therefore requires

$$|1 - \eta\lambda_i| < 1 \quad \forall i \Leftrightarrow 0 < \eta < \frac{2}{\lambda_{max}} \tag{5.7}$$

and the slowest mode, controlled by the smallest eigenvalue, decays as $(1 - \eta\lambda_{min})^k$. This produces the two classical facts:

$$\text{condition number } \kappa = \frac{\lambda_{max}}{\lambda_{min}}, \qquad \text{iterations} \propto \kappa \log\frac{1}{\varepsilon} \tag{5.8}$$

An ill-conditioned problem (large κ) converges slowly even with the best η — the direct motivation for momentum, adaptive methods, and for PCA preprocessing (Question 8).

Effect of the learning rate

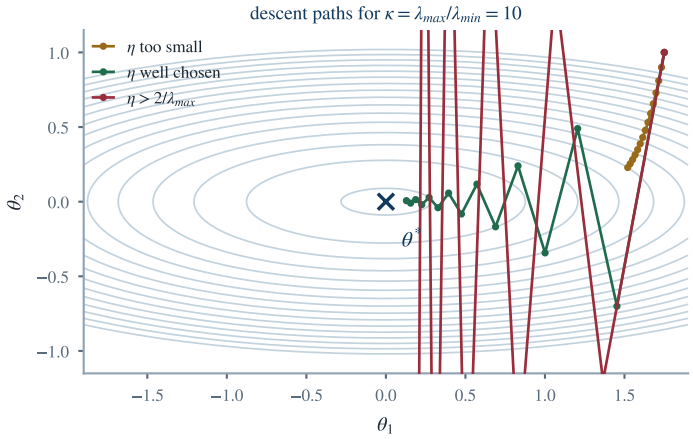


Fig. 5.1 Descent paths on a quadratic with condition number 10. A small rate crawls, a well-chosen rate converges steeply, and a rate above $2/\lambda_{max}$ oscillates and diverges.

- **η too small ($\eta \ll 1/L$).** Every step is safe, so J decreases monotonically, but each step makes only a tiny dent: convergence takes very many iterations, training appears "stuck", and finite iteration budgets end far from the minimum. Gradient noise is also under-damped relative to progress.
- **η too large ($\eta > 2/\lambda_{max}$).** The step overshoots the valley: J oscillates from wall to wall, then grows geometrically, and the parameters (and the loss) blow up to *inf*/NaN. In practice one sees a loss that jumps up and down or a sudden divergence after a few hundred steps.
- **η well chosen ($\eta \approx 1/L$).** Fast monotone decrease, no oscillation; this is the target, and it is usually found by a logarithmic search over $\{10^{-1}, 3\times10^{-2}, \ldots, 10^{-5}\}$.

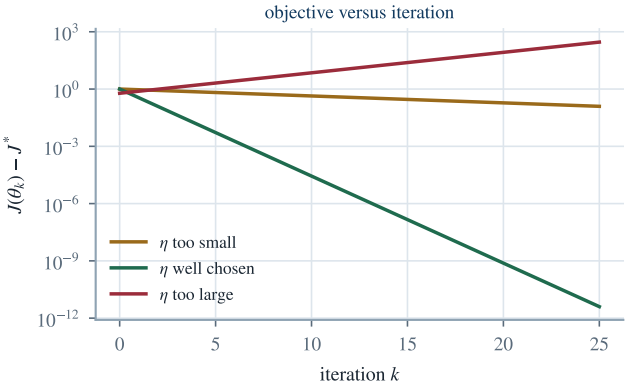


Fig. 5.2 Objective versus iteration for three learning rates on a log scale: too small (slow linear decay), well chosen (fast decay), too large (divergence).

Practical consequences of the analysis

1. **Scale the learning rate with curvature:** for batch size B, a common heuristic is $\eta \propto B$ (linear scaling) with a short warm-up.
2. **Decay η over time:** $\eta_t = \eta_0/(1 + t/T)$ or cosine decay, so that late training behaves like a small-step refinement.
3. **Precondition the problem:** normalising inputs, batch normalisation and PCA reduce κ, which effectively buys a larger safe η.
4. **Clip the gradient** so that an occasional huge ∇J cannot act like a temporary enormous learning rate.
5. **Monitor the loss curve:** a monotone but slow curve means η is small; a saw-tooth or divergent curve means η is large.

Algorithm

1. Initialise θ_0 (Xavier/He) and set η, tolerance ε, maximum iterations T.

2. Compute the full gradient $\nabla J(\theta_k) = 1/m \Sigma_i \nabla \ell(f(x^{(i)}; \theta_k), y^{(i)})$.
3. Update $\theta_{k+1} = \theta_k - \eta \nabla J(\theta_k)$.
4. Stop when $\|\nabla J(\theta_k)\| < \varepsilon$ or $k = T$; else repeat from step 2.

MARKS ARE LOST BY

Writing only "$\theta = \theta - \eta \nabla J$" with no derivation, ignoring the condition $0 < \eta < 2/\lambda_{max}$, and failing to distinguish *slow convergence* (small η) from *divergence* (large η). Show the Taylor expansion and the $(I - \eta H)$ recursion — those are the analytical parts of the question.

GOLDEN LINE

The update is $\theta \leftarrow \theta - \eta \nabla J(\theta)$ because $-\nabla J$ is the steepest-descent direction. Convergence needs $\eta < 2/\lambda_{max}$; too small is slow, too large diverges, and $\kappa = \lambda_{max}/\lambda_{min}$ decides the speed.

KEYWORDS TO WRITE IN THE EXAM

Gradient descent · steepest descent · Taylor expansion · learning rate · L-smoothness · step size $1/L$ · convergence condition · condition number · oscillation · divergence · learning-rate schedule

Q6 Formulate a constrained optimisation problem for minimising a model objective subject to a parameter-norm constraint. Explain the relationship between constrained optimisation and regularisation.

CO1 · L5 (Design) · 10 Marks

Formulation

Unconstrained training may choose parameters with an enormous norm, which fits noise and destroys generalisation. A natural cure is to **forbid** such parameters outright:

$$\min_{\theta} J(\theta) \quad \text{subjectto} \quad \|\theta\|_2 \le c, \qquad c > 0 \tag{6.1}$$

The feasible set $\mathcal{C} = \{\theta : \|\theta\|_2 \le c\}$ is a **closed convex ball**; J is the training loss (for example the mean squared error). Because the constraint set is convex and J is differentiable, a global minimum over $\mathcal{C}$ exists and the problem is well posed even when J is not convex globally.

Lagrangian and KKT conditions

Attach a multiplier $\lambda \ge 0$ to the constraint and write

$$\mathcal{L}(\theta, \lambda) = J(\theta) + \lambda\left(\|\theta\|_2^2 - c^2\right) \tag{6.2}$$

The Karush–Kuhn–Tucker conditions for this problem are

$$\nabla J(\theta^*) + 2\lambda\theta^* = 0, \qquad \lambda \ge 0, \qquad \lambda\left(\|\theta^*\|_2^2 - c^2\right) = 0 \tag{6.3}$$

The third condition (**complementary slackness**) splits the analysis into exactly two cases:

1. **Inactive constraint** ($\lambda = 0$): the unconstrained minimiser already satisfies $\|\theta^*\| < c$. Then $\nabla J(\theta^*) = 0$ and the constraint has no effect. This happens for small models or large c.
2. **Active constraint** ($\lambda > 0$): the unconstrained minimiser lies outside the ball, so the solution sits **on the boundary** $\|\theta^*\| = c$, with $\nabla J(\theta^*) = -2\lambda\theta^*$. At that point the gradient of the loss is *parallel* to the outward normal θ^* of the ball: the loss can no longer be reduced without leaving the allowed region.

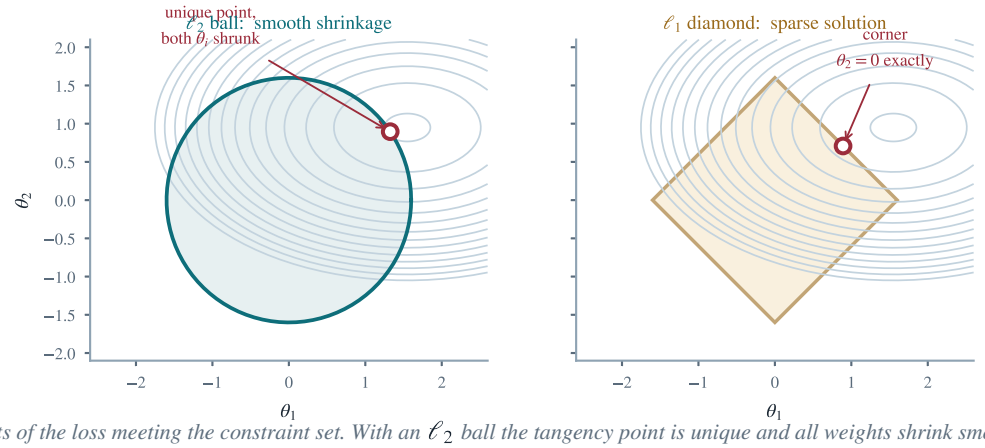


Fig. 6.1 Level sets of the loss meeting the constraint set. With an ℓ_2 ball the tangency point is unique and all weights shrink smoothly; with an ℓ_1 diamond the contact almost always occurs at a corner, so one weight becomes exactly zero.

Geometric interpretation

Draw the level curves $J(\theta) =$ const and grow the ball from the origin. The constrained optimum is the first point where a level curve touches the feasible set. Geometrically this is a **tangency condition**: the loss gradient is orthogonal to the constraint surface. That tangency is precisely what the KKT stationarity equation states.

From constraint to penalty (the equivalence)

Define the **penalised (regularised)** problem

$$\min_{\theta} J(\theta) + \lambda \|\theta\|_2^2 \tag{6.4}$$

Its stationarity condition is $\nabla J(\theta) + 2\lambda\theta = 0$ — **identical** to the KKT condition of the constrained problem with the same λ. Therefore, for every radius c there exists a multiplier $\lambda^*(c) \geq 0$ such that

$$\arg\min_{\|\theta\| \leq c} J(\theta) = \arg\min_{\theta} J(\theta) + \lambda^*(c)\|\theta\|_2^2 \tag{6.5}$$

and the map $c \mapsto \lambda^*$ is monotonically decreasing: a tighter constraint (smaller c) needs a larger penalty (larger λ). Conversely, for every λ the penalised solution lies on the boundary of some ball of radius $c(\lambda) = \|\theta_\lambda^*\|$. This is the **Lagrangian duality** that justifies ridge regression and weight decay: the constrained and the penalised formulations describe the same family of solutions.

$$\text{tighter ball } c \downarrow \quad \Longleftrightarrow \quad \text{stronger penalty } \lambda \uparrow \tag{6.6}$$

Two norms, two kinds of shrinkage

Aspect	ℓ_2 constraint / ridge penalty	ℓ_1 constraint / lasso penalty
Feasible set	sphere / ball (smooth)	diamond / cross-polytope (corners)
Solution location	tangency at a smooth point — unique and stable	usually a corner: some $\theta_j = 0$ exactly
Effect on weights	shrinks all weights smoothly, none becomes zero	performs variable selection (sparse model)
Gradient	differentiable everywhere ($2\lambda\theta$)	non-differentiable at 0 (needs subgradients / soft thresholding)
Closed form (linear model)	$w = (X^\top X + \lambda I)^{-1} X^\top y$	no closed form; $\text{sign}(w)\max(\lvert w\rvert - \lambda, 0)$ after orthonormalisation
Typical use	default regulariser, stabilises ill-conditioned problems	interpretable, compressed models

Ridge has a revealing spectral form. If $X^\top X = Q\Lambda Q^\top$, then

$$w_{\text{ridge}} = Q\,\text{diag}\left(\frac{\lambda_i}{\lambda_i + \alpha}\right) Q^{\top} w_{\text{OLS}} \tag{6.7}$$

so each eigen-direction is shrunk by the factor $\lambda_i/(\lambda_i + \alpha)$: directions with small variance (small λ_i) are shrunk most, which is exactly why ridge stabilises ill-conditioned problems and why $\alpha > 0$ always makes $X^{\top}X + \alpha I$ invertible.

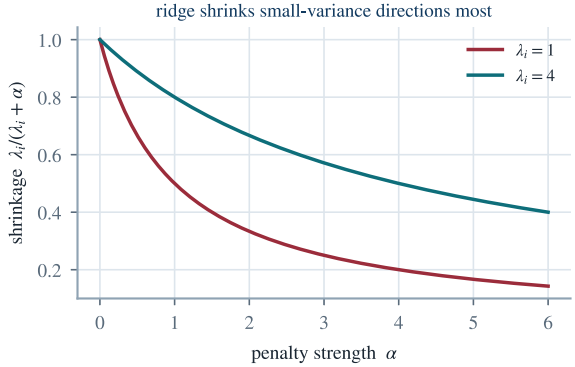


Fig. 6.2 Ridge shrinkage factor per eigen-direction. Small-variance directions are suppressed most, so the effective capacity of the model falls as the penalty grows.

Why this improves generalisation

- **Restricting capacity.** Limiting $\|\theta\|$ limits the complexity of the representable functions, which lowers variance at the cost of a small increase in bias — the bias–variance trade-off performed on purpose.
- **Suppressing noise directions.** Directions with little data support get large variance in the unconstrained solution; shrinkage removes them.
- **Numerical stability.** αI regularises the spectrum of $X^{\top}X$, so the solution no longer amplifies small singular values.
- **Implicit equivalence.** Early stopping, small initialisation and dropout are also forms of norm restriction: they keep the parameters near a small-norm region, which is why they behave like regularisers.

Design procedure

1. Standardise features, so the norm constraint is not dominated by units.
2. Choose the regulariser family: ℓ_2 (default), ℓ_1 (sparsity) or elastic net ($\ell_1 + \ell_2$).
3. Select λ (equivalently c) **on validation data**, searching logarithmically, e.g. $\lambda \in \{10^{-4}, \ldots, 10^{1}\}$.
4. Train with the penalty inside the objective — never by clipping weights after the update, which changes the descent dynamics.
5. Verify that the training and validation curves move together; a large gap still remaining means λ is too small.

ONE LINE THAT EARNS THE MARKS

"The constrained problem $\min J(\theta)$ s.t. $\|\theta\| \le c$ has KKT condition $\nabla J + 2\lambda\theta = 0$, which is the same as the stationarity condition of the penalty problem $\min J(\theta) + \lambda\|\theta\|^2$; hence a norm constraint and a norm penalty are two views of the same restriction, with c and λ inversely related."

GOLDEN LINE

A norm constraint caps model capacity; its KKT multiplier becomes the penalty weight. ℓ_2 shrinks smoothly, ℓ_1 also sparsifies, and both are the same idea as seen from the constraint side and the penalty side.

KEYWORDS TO WRITE IN THE EXAM

Constrained optimisation · feasible set · Lagrangian · KKT conditions · complementary slackness · active/inactive constraint · norm constraint · regularisation · ridge · lasso · sparsity · weight decay · duality

Q7 **For a least-squares problem $J(w) = \|Xw - y\|^2$, derive the normal equations and discuss the conditions under which the solution is unique.**

CO1 · L4 (Analyse) · 10 Marks

Problem statement

Given a design matrix $X \in \mathbb{R}^{m \times n}$ (m samples, n features) and targets $y \in \mathbb{R}^m$, find the weight vector that minimises the sum of squared errors:

$$J(w) = \|Xw - y\|_2^2 = (Xw - y)^\top (Xw - y) = \sum_{i=1}^{m} \left(x^{(i)\top} w - y^{(i)}\right)^2 \tag{7.1}$$

This is the loss of a **linear model with a squared-error criterion**, and it is also the negative log-likelihood of a Gaussian noise model (Question 5 of Unit II).

Derivation of the normal equations

Expand the objective:

$$J(w) = w^\top X^\top X w - 2 w^\top X^\top y + y^\top y \tag{7.2}$$

Differentiate with respect to the vector w (using $\frac{\partial a^\top w}{\partial w} = a$ and $\frac{\partial w^\top A w}{\partial w} = 2Aw$ for symmetric A):

$$\frac{\partial J}{\partial w} = 2X^\top X w - 2X^\top y \tag{7.3}$$

Setting the gradient to zero gives the **normal equations**:

$$X^\top X w = X^\top y \qquad \Longrightarrow \qquad \hat{w} = (X^\top X)^{-1} X^\top y \tag{7.4}$$

(Second-order check: $\nabla^2 J = 2X^\top X$ is positive semi-definite, so any stationary point is a global minimum — J is convex.)

Geometric derivation (orthogonality principle)

Xw is a linear combination of the columns of X, so it ranges over the **column space** $\mathrm{col}(X)$. Minimising $\|Xw - y\|$ means finding the point of that subspace closest to y — the orthogonal projection. At the optimum the residual must be perpendicular to every column of X:

$$X^\top (y - X\hat{w}) = 0 \Leftrightarrow X^\top X \hat{w} = X^\top y \tag{7.5}$$

Both derivations give the same equations; the geometric one explains *why* they are correct.

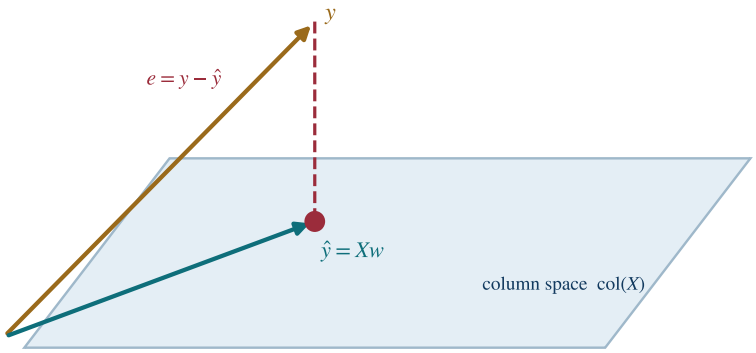


Fig. 7.1 Least squares as an orthogonal projection: $\hat{y} = Xw$ is the closest point of col(X) to y, and the residual $e = y - \hat{y}$ is orthogonal to the whole subspace.

The projection (hat) matrix

$$\hat{y} = X(X^\top X)^{-1}X^\top y = Hy, \qquad H = X(X^\top X)^{-1}X^\top \tag{7.6}$$

H is symmetric ($H^\top = H$) and **idempotent** ($H^2 = H$): projecting twice changes nothing. Its trace equals n, the dimension of the subspace, and $I - H$ projects onto the residual space. The minimum value of the objective is $\|(I - H)y\|^2$.

Uniqueness of the solution

The solution exists and is **unique if and only if** $X^\top X$ **is invertible**, which for an $m \times n$ matrix is equivalent to the following conditions:

$$\text{rank}(X) = n \Leftrightarrow \text{columns of } X \text{ are linearly independent} \Leftrightarrow \text{null}(X) = \{0\} \tag{7.7}$$

Equivalently, all n singular values of X are non-zero, so $\kappa(X) < \infty$. This requires $m \geq n$ (at least as many samples as features) *and* no redundancy among the features.

Situation	Rank / shape	Consequence for $X^\top X$	Nature of the solution
Well-posed (tall, independent)	$m \geq n$, rank n	invertible, κ finite	unique $\hat{w} = (X^\top X)^{-1}X^\top y$
Under-determined (wide)	$m < n$	singular, rank $\leq m < n$	infinitely many exact solutions ($J = 0$)
Collinear / duplicate features	rank $< n$	singular	infinitely many solutions, same $\hat{y}$
Constant column + intercept	rank deficiency	singular	intercept and slope not identifiable separately
Ridge-regularised	any shape	$X^\top X + \alpha I \succ 0$	always unique: $(X^\top X + \alpha I)^{-1}X^\top y$

When the solution is not unique, the set of minimisers is an affine subspace, and the standard choice is the **minimum-norm** solution obtained from the Moore–Penrose pseudo-inverse:

$$\hat{w} = X^+ y = V\Sigma^{-1}U^\top y, \qquad \hat{w} = \arg\min\{\|w\|_2 : Xw = \hat{y}\} \tag{7.8}$$

Note that $\hat{y} = X\hat{w}$ remains unique even when $\hat{w}$ does not: the projection onto a subspace is always unique.

Numerical considerations (analysis marks)

1. **Never invert $X^\top X$ directly.** Forming $X^\top X$ squares the condition number, $\kappa(X^\top X) = \kappa(X)^2$, so half of the significant digits can be lost.
2. **Use QR:** factor $X = QR$ with $Q^\top Q = I$ and R upper triangular; then $X^\top X = R^\top R$ and the normal equations reduce to the triangular system $R\hat{w} = Q^\top y$, solved by back substitution in $O(n^2)$ with condition number $\kappa(R) = \kappa(X)$.
3. **Use SVD for rank-deficient or ill-conditioned data:** the pseudo-inverse truncates negligible singular values, giving a stable minimum-norm answer.
4. **Add ridge** ($\alpha > 0$) when features are collinear; this guarantees uniqueness and bounds the solution norm.

Worked example

Fit $y = w_0 + w_1 x$ to the three points $(0, 1), (1, 2), (2, 4)$, so that X has columns $(1, 1, 1)^\top$ and $(0, 1, 2)^\top$ and $y = (1, 2, 4)^\top$.

$$X^\top X = \begin{bmatrix} 3 & 3 \\ 3 & 5 \end{bmatrix}, \qquad X^\top y = \begin{bmatrix} 7 \\ 11 \end{bmatrix} \tag{7.9}$$

Solving the two equations in w gives $w_1 = 1.5$, $w_0 = 0.833$, so the fitted line is $\hat{y} = 0.833 + 1.5x$ and the residual sum of squares is 0.167. The matrix $X^\top X$ has determinant $3 \cdot 5 - 9 = 6 \neq 0$, so the fit is unique — the two columns are not collinear.

EXAMINER'S CHECKLIST

(i) Expand J, (ii) differentiate to $2X^\top Xw - 2X^\top y$, (iii) state $X^\top Xw = X^\top y$, (iv) give $\hat{w} = (X^\top X)^{-1}X^\top y$, (v) prove minimality via positive semi-definiteness **or** the orthogonality principle, (vi) state the uniqueness condition $\text{rank}(X) = n$, and (vii) mention QR/SVD for numerical stability.

GOLDEN LINE

Least squares is an orthogonal projection of y onto $\text{col}(X)$; setting the residual orthogonal to that subspace gives the normal equations $X^\top Xw = X^\top y$. The solution is unique exactly when X has full column rank.

KEYWORDS TO WRITE IN THE EXAM

Least squares · normal equations · residual · orthogonal projection · hat matrix · idempotent · full column rank · uniqueness · under-determined · pseudo-inverse · QR decomposition · condition number · ridge

Q8 **A feature matrix contains highly correlated variables. Evaluate whether PCA is appropriate for preprocessing before a neural network, and justify the decision using variance and decorrelation.**

CO1 · L5 (Design) · 10 Marks

The problem with correlated inputs

Suppose the input matrix has features that are almost linear combinations of each other, so the covariance matrix C has large off-diagonal entries and several small eigenvalues. For a network this causes three concrete problems.

1. **Redundant parameters.** The first layer spends capacity learning to disentangle directions that carry the same information.
2. **Ill-conditioned optimisation.** The Hessian of the loss inherits the spectrum of C. With $\kappa = \lambda_{max}/\lambda_{min} \gg 1$, gradient descent needs about $\kappa \log(1/\varepsilon)$ iterations (Question 5): the loss surface is a narrow valley and training is slow.
3. **Unstable weights.** Collinear inputs make the optimal weights non-unique; different runs give very different but equally accurate solutions, and the weights change violently under small perturbations of the data.

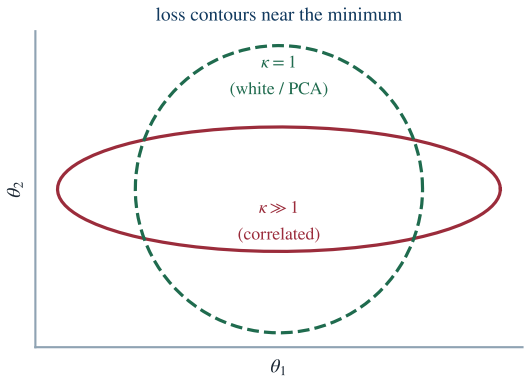


Fig. 8.1 Covariance (hence loss) contours of a correlated problem versus a decorrelated one. Round contours mean every direction has similar curvature, so one learning rate suits all parameters.

What PCA does to the inputs

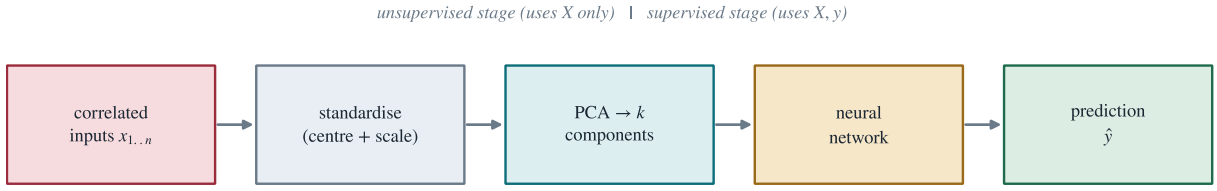


Fig. 8.2 Where PCA sits in the pipeline: an unsupervised reduction stage followed by the supervised network.

1. Centre the training data and compute $C = 1/(m-1)X^\top X$, with eigenvalues $\lambda_1 \geq \ldots \geq \lambda_n$.
2. Factorise $C = W\Lambda W^\top$ and keep the top k eigenvectors W_k.
3. Project $Z = XW_k$. Then $\mathrm{Cov}(Z) = W_k^\top CW_k = \Lambda_k$ is **diagonal**: the new inputs are uncorrelated, with variances $\lambda_1, \ldots, \lambda_k$.
4. Optionally **whiten** so that all directions have unit variance: $Z_{\text{white}} = XW_k\Lambda_k^{-1/2}$, giving $\mathrm{Cov}(Z_{\text{white}}) = I$ and $\kappa = 1$.

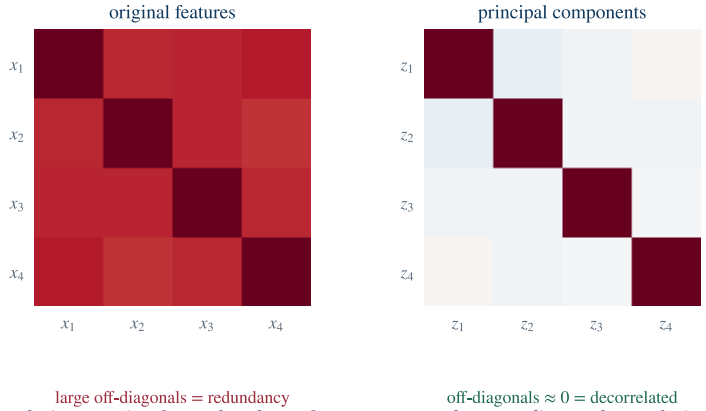


Fig. 8.3 Before and after: the correlation matrix of correlated raw features versus the near-diagonal correlation matrix of principal components.

Benefits for the network

Effect	Mechanism	Practical consequence
Decorrelation	$\mathrm{Cov}(Z) = \Lambda_k$ diagonal	round loss contours, one learning rate suits all weights
Better conditioning	κ falls from $\lambda_{max}/\lambda_{min}$ to λ_{max}/λ_k (to 1 after whitening)	fewer iterations to reach the same loss
Fewer inputs	$n \to k$	first-layer parameters fall from $(n+1)h$ to $(k+1)h$; less memory and compute
Noise removal	low-variance directions discarded	small random fluctuations are suppressed, which can improve generalisation
Redundancy removal	components are orthogonal	no duplicated information to disentangle
Visualisation	scores z_1, z_2	data can be plotted and outliers inspected

The parameter saving is easy to quantify: a first hidden layer of $h = 128$ units on $n = 256$ correlated inputs needs $257 \times 128 = 32{,}896$ weights; after PCA to $k = 40$ it needs $41 \times 128 = 5{,}248$ — an 84% reduction with almost no variance lost.

The risks — when PCA is the wrong choice

These determine whether the answer is "yes" or "no", and they carry the evaluation marks.

1. **Variance is not relevance.** PCA is *unsupervised*: it keeps the directions with the largest variance, which need not be the directions that predict y. A low-variance feature that is highly predictive can be thrown away. This is a real loss of accuracy, not a theoretical worry.

2. **Linearity.** PCA only removes *linear* correlation. If the informative structure is nonlinear, an autoencoder or kernel PCA is needed.
3. **Scale dependence.** The variance ordering is meaningless if features have different units, so standardisation must precede PCA.
4. **Loss of interpretability.** Each component is a mixture of all original features, so a model built on components cannot be explained feature-by-feature (a problem in clinical or credit-risk settings).
5. **Leakage.** The mean, scaling and W_k must be fitted on the **training split only** and then applied to validation and test data; otherwise the test information leaks into training.
6. **Deeper networks already adapt.** A stacked network can learn its own linear mixture in the first layer, so the benefit is largest for shallow or small models and for classical classifiers (SVM, logistic regression) attached to the features.

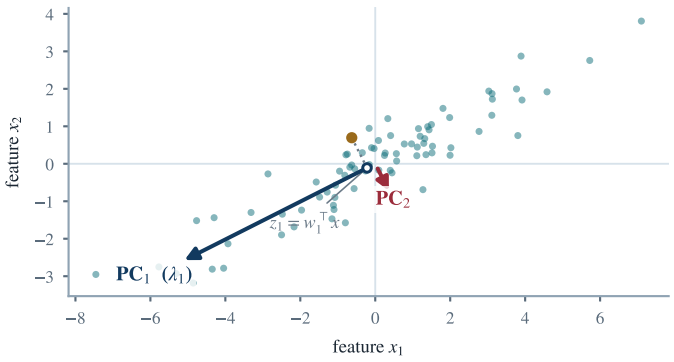


Fig. 8.4 What is kept and what is lost: the spread along w_1 is preserved, while the (small) spread along w_2 is discarded — correct only if the discarded direction is irrelevant.

Decision procedure

1. Standardise the features and inspect the correlation matrix C (or R). Off-diagonals near ± 1 confirm strong linear redundancy.
2. Check the spectrum: if one or two eigenvalues dominate (e.g. $\text{Cum}_2 > 0.95$), the data is intrinsically low-dimensional and PCA is appropriate.
3. Choose k by **cross-validation of the final network accuracy**, not by variance alone — and compare against the un-reduced baseline.
4. Fit W_k on the training fold only; apply the same transformation to validation and test data. Re-fit inside every cross-validation fold.
5. Compare three pipelines: (a) no PCA, (b) PCA with 95% variance, (c) PCA with whitening. Keep the one with the best validation score at comparable cost.
6. If the discarded low-variance directions turn out to be predictive (accuracy drops), abandon PCA and use feature selection, an autoencoder, or a network with more capacity instead.

Verdict

For a feature matrix with **high linear correlation** ($|\rho| > 0.9$), a **skewed spectrum** and a **shallow or classical model**, PCA is appropriate: it decorrelates the inputs, improves conditioning, saves parameters and can denoise. For **nonlinear** structure, for problems where **every direction may matter** (small k, weak signal), or where **interpretability is required**, PCA should not be applied blindly; an autoencoder or feature selection is preferable. The defensible answer is therefore conditional, and must be justified by the covariance spectrum and confirmed by validation accuracy.

MAKE THE EVALUATION EXPLICIT

The question asks you to *evaluate*, so a one-sided "PCA is always good" answer caps the marks. Give the spectrum argument for, the "variance $\neq$ relevance" argument against, and close with the cross-validated decision rule.

GOLDEN LINE

PCA before a network buys decorrelation, better conditioning and fewer parameters, at the risk of discarding low-variance but predictive directions. Decide from the covariance spectrum and confirm by validation accuracy.

KEYWORDS TO WRITE IN THE EXAM

PCA preprocessing · correlated features · decorrelation · covariance spectrum · conditioning · whitening · parameter reduction · variance $\neq$ relevance · data leakage · autoencoder alternative

Rapid revision — the whole unit in one grid

Read this column-wise the night before the examination: the trigger phrase, the three things that must appear, and the trap that costs marks.

Question	Three things that must appear	The trap
1 PCA	centring; $C = W\Lambda W^\top$; eigen $=$ variance; $Z = XW_k$; cumulative-variance rule	forgetting to centre the data
2 Eigen vs SVD	$X = U\Sigma V^\top$; $X^\top X = V\Sigma^2 V^\top$; $\lambda_i = \sigma_i^2/(m-1)$	saying the two routes give different PCs
3 Overflow/underflow	float32 limits; max-shift; log-domain; gradient clipping; NaN sources	"just use float64" without a mathematical fix
4 Moments	$\mathbb{E}$, Var, Cov; covariance matrix; trace $=$ total variance; $Cw = \lambda w$	claiming $\mathrm{Cov} = 0$ implies independence
5 Gradient descent	Taylor/steepest-descent derivation; $0 < \eta < 2/\lambda_{max}$; $(I - \eta H)^k$ recursion	no derivation, only the update rule
6 Constrained optimisation	Lagrangian, KKT, complementary slackness; $\nabla J + 2\lambda\theta = 0$; constraint $\Leftrightarrow$ penalty	confusing ℓ_1 and ℓ_2 geometry
7 Least squares	expansion $\rightarrow$ gradient; $X^\top Xw = X^\top y$; orthogonality; rank condition	inverting $X^\top X$ when it is singular
8 PCA before a network	decorrelation, conditioning, parameter saving; variance $\neq$ relevance; CV decision	one-sided "PCA is always good"

G. PULLAIAH COLLEGE OF ENGINEERING AND TECHNOLOGY
Autonomous · Affiliated to JNTUA, Anantapur · Kurnool

A40531 · DEEP LEARNING · IV B.Tech CSE · I Semester

UNIT II

Model Answers for the 10-Mark Question Bank

Derivations · formulas · diagrams · examination keywords

Machine-Learning Basics and Deep Feedforward Networks

Q1 Bias and variance diagnosis from training and validation error

Q2 Validation strategy without contaminating the test set

Q3 Batch gradient descent versus SGD for large networks

Q4 A model fails on unseen data: the first intervention

Q5 Maximum likelihood, output units and losses

Q6 Design of a multiclass feedforward network

Q7 Depth, width, hidden units and representational capacity

Q8 End-to-end training workflow for a fully connected network

Every answer is written to the 10-mark pattern: definition, derivation, formula, diagram, comparison table and conclusion, followed by the keywords that examiners look for.

08
MODEL ANSWERS

49
NUMBERED FORMULAE

14
VECTOR DIAGRAMS

Contents

Unit II — Machine-Learning Basics and Deep Feedforward Networks

Q1. Bias and variance diagnosis 3

Q2. Validation strategy 5

Q3. Batch GD vs SGD 8

Q4. First intervention 11

Q5. Maximum likelihood and loss 13

Q6. Multiclass feedforward network 15

Q7. Depth, width and capacity 18

Q8. End-to-end training workflow 21

The unit at a glance

Q	Core idea	Result to memorise
1	Two error numbers decide everything	gap $= E_{val} - E_{train}$; large gap $\Rightarrow$ variance
2	Parameters, hyperparameters and the estimate need separate data	train $\rightarrow$ parameters, val $\rightarrow$ hyperparameters, test once
3	Noise variance falls as $1/B$, updates grow as m/B	$\mathrm{Var}(\hat{g}) = \sigma^2/B$
4	A low training error rules out the capacity explanation	first fix variance: data, augmentation, regularisation
5	Pick the distribution, then the loss and output unit follow	Gaussian → MSE, Bernoulli → BCE, categorical → CE
6	Softmax plus categorical cross-entropy is forced by the task	$\hat{p} = \mathrm{softmax}(z)$, $J = -\Sigma_k y_k \log \hat{p}_k$
7	Depth buys exponential expressivity, width polynomial	regions $\propto h^L$; parameters linear in L
8	Split, fit on train, then loop forward–loss–backward–update	$\theta \leftarrow \theta - \eta \nabla_\theta J$

HOW TO USE THIS BOOKLET

Each answer follows the same skeleton, so you can revise the *pattern* once and then recall the content: **idea → derivation → formula → diagram → table → conclusion**.

Formula boxes carry equation numbers such as (3.2) — that is *Question 3, second equation*. Figure numbers follow the same rule, so (Fig. 5.1) belongs to Q5.

The **Golden line** is the one sentence to write at the end of an answer; the **Keywords** block is the list an examiner scans for. Aim to reproduce the boxed formulae exactly and to reproduce the diagrams approximately.

Q1 **A model has very low training error but high validation error. Analyse the likely causes using capacity, bias and variance, and propose a systematic diagnosis.**

CO2 · L5 (Design) · 10 Marks

Reading the two numbers

Two measurements decide everything: the **training error** E_{train} and the **validation error** E_{val} on data the model never saw. Their *levels* and their *gap* separate the three possible situations.

$$\text{gap} = E_{val} - E_{train}, \qquad \text{capacity of the hypothesis class } \mathcal{H} \tag{1.1}$$

Observation	Diagnosis	Dominant term
E_{train} high, E_{val} high, small gap	underfitting	bias
E_{train} very low, E_{val} high, large gap	overfitting	variance
E_{train} low, E_{val} low, small gap	good fit	both acceptable

The case in the question is the **second row**: the network has enough capacity to memorise the training set, so it has learned the noise as well as the signal.

Formal decomposition

For a target $y = f(x) + \varepsilon$ with $\mathbb{E}[\varepsilon] = 0$ and $\mathrm{Var}(\varepsilon) = \sigma^2$, the expected squared error of the trained model $\hat{f}$ at a point x decomposes as

$$\mathbb{E}\left[(y - \hat{f}(x))^2\right] = \left(f(x) - \mathbb{E}\hat{f}(x)\right)^2 + \mathrm{Var}\left(\hat{f}(x)\right) + \sigma^2 \tag{1.2}$$

- **Bias** is the error of the average model: how far the family $\mathcal{H}$ is from the truth. It is reduced by *more capacity*.
- **Variance** is the sensitivity of the fitted model to the particular training sample. Models with many parameters, large weights or long training have high variance.
- **Noise** σ^2 is irreducible; no model can remove it, which is why E_{val} can never reach zero.

Because a model with very low E_{train} has, by construction, small bias on the training points, the observed failure must be **variance**: a different training sample would produce a very different network, and the average of those networks would be much better than any single one.

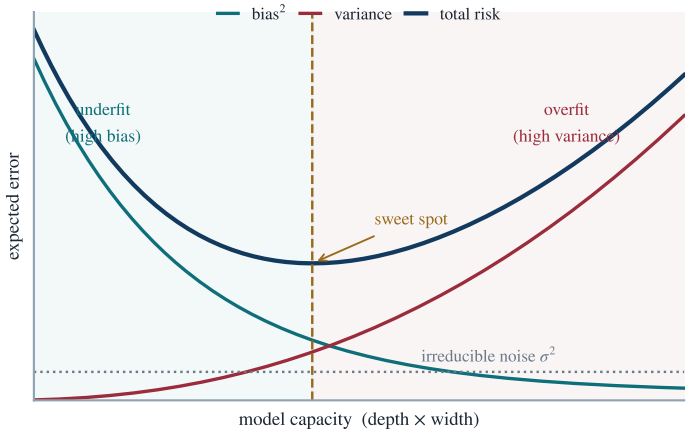


Fig. 1.1 Bias–variance decomposition as a function of capacity. The total risk is U-shaped; overfitting is the right-hand branch.

Why low training error does not imply a good model

1. **Memorisation is cheaper than generalisation.** With p parameters and m samples, a large network can interpolate the training set exactly (even random labels) while learning no transferable structure.
2. **Capacity exceeds data.** The number of parameters is large relative to m, so many parameter settings fit the training data; gradient descent lands in one of them, chosen by the noise it happened to see.

3. **Training error is an optimistically biased estimate.** It is measured on exactly the data used to choose the parameters, so it is the *minimum* of the loss over the sampled set, not its expectation.
4. **Long training with no regularisation** lets weights grow, and large weights amplify input differences — the formal signature of high variance.

Diagnostic procedure

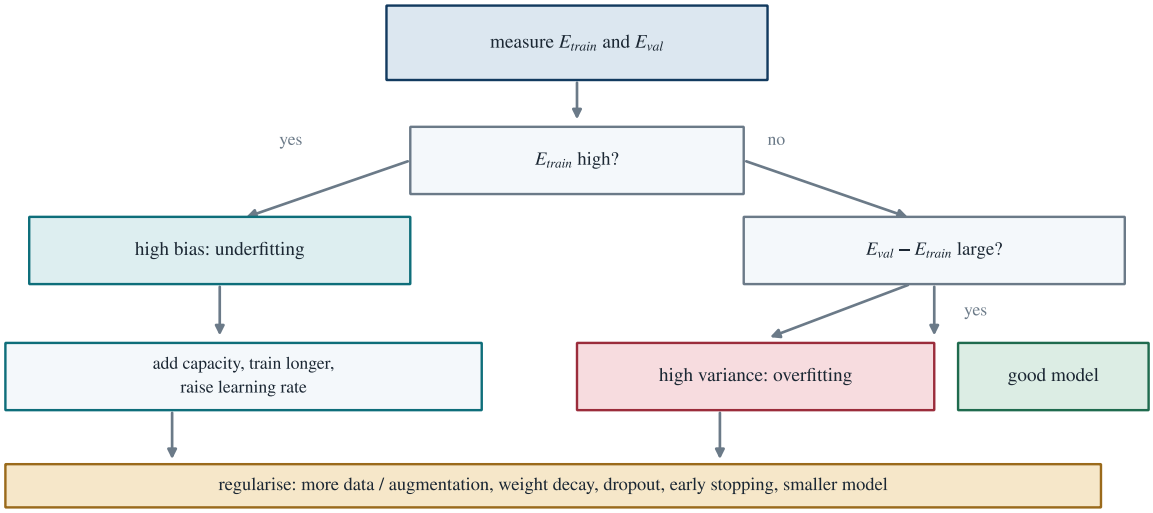


Fig. 1.2 Decision tree from the two error measurements to the appropriate family of interventions.

1. **1 — Verify the measurement.** Recompute both errors with the same preprocessing, the same loss and *model.eval()* (dropout and batch-norm behave differently in training mode). A "validation" error computed with the network in training mode is meaningless.
2. **2 — Check for leakage.** If validation accuracy is suspiciously *high*, features derived from the target, duplicate rows or preprocessing fitted on the whole dataset may be the cause. Confirm that the mean, scaling and PCA used for validation were fitted on the training split only.
3. **3 — Compute the gap explicitly.** A gap of a few percent can be normal; a gap where $E_{val} \approx 2\text{–}3 \times E_{train}$ is a genuine variance problem.
4. **4 — Plot learning curves** of E_{train} and E_{val} against training-set size.

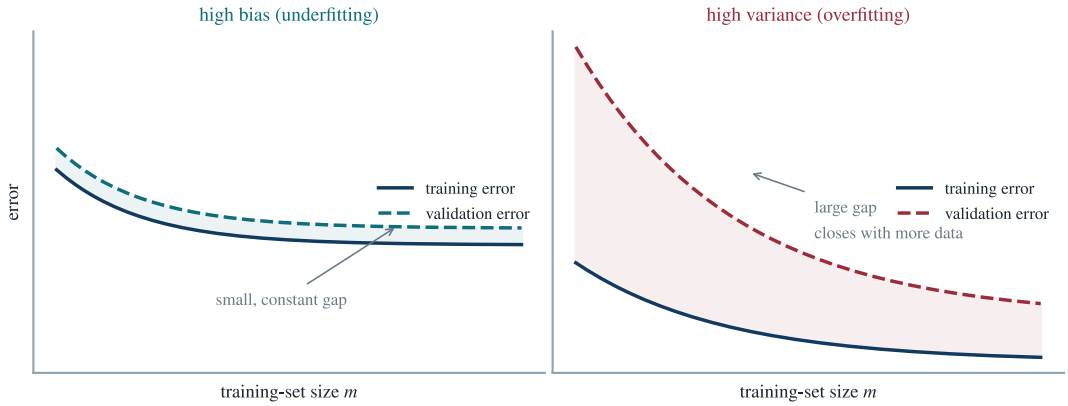


Fig. 1.3 Left: high bias — both curves plateau high with a small gap. Right: high variance — the curves are far apart and the gap closes as data is added.

1. **5 — Test the capacity hypothesis.** Deliberately shrink the model (fewer units/layers) or train with strong weightdecay; if E_{val} improves while E_{train} worsens, variance is confirmed.
2. **6 — Test the data hypothesis.** Train on a subset (e.g.\ 25\%) and on the full set. If the gap narrows when data is added, the diagnosis is "insufficient data for this capacity".
3. **7 — Cross-check with k-fold cross-validation.** A large standard deviation of the validation score across folds is direct evidence of high variance.

Remedies matched to the diagnosis

| Lever | Effect on bias | Effect on variance | Typical setting |

| More training data / augmentation | none | strong decrease | the first intervention when possible |

| Weight decay (ℓ_2) | small increase | decrease | $\lambda \in [10^{-5}, 10^{-2}]$ |

| Dropout | small increase | decrease | 0.2–0.5 on wide layers |

| Early stopping | small increase | decrease | stop at minimum validation loss |

| Smaller architecture | increase | decrease | cut width before depth |

| More capacity | decrease | **increase** | only if E_{train} is also high |

| Longer training | decrease | **increase** | only for underfitting |

Worked example

A $784 \to 512 \to 256 \to 10$ network on 5,000 training images reaches $E_{train} = 0.4\%$ and $E_{val} = 14\%$. The gap of 13.6 percentage points with a near-zero training error is the textbook signature of **high variance**: the model has enough capacity to store the training set.

1. Confirmation: 5-fold cross-validation gives $13.1\% \pm 3.8\%$ — a large spread, again variance.
2. First action: data augmentation and weight decay 10^{-4}; E_{train} rises to 1.8%, E_{val} falls to 8.5%.
3. Second action: add dropout 0.3 after each dense layer and stop early at the best validation epoch: $E_{val} = 6.9\%$.
4. Architecture is *not* reduced, because reducing capacity is only justified once the gap has been reduced by cheaper means.

HOW TO STRUCTURE THE ANSWER

State the two numbers, compute the gap, name the diagnosis (high variance), then give the decomposition formula with all three terms, then the diagnostic procedure, then the matched remedies. Naming the wrong direction ("add capacity") is the commonest way to lose the analytical marks.

GOLDEN LINE

Low training error with high validation error means high variance, not high bias. Diagnose with the gap and the learning curves, then reduce capacity via data, regularisation and early stopping — never by adding more parameters.

KEYWORDS TO WRITE IN THE EXAM

Bias · variance · irreducible noise · bias–variance decomposition · capacity · generalisation gap · learning curve · cross-validation · overfitting · underfitting · weight decay · early stopping

Q2 Design a validation strategy for selecting learning rate, hidden-layer size and number of layers without contaminating the test set. Justify every step.

CO2 · L5 (Design) · 10 Marks

Principle

Three different questions need three different data sources. Mixing them is the single most common experimental error.

$$\text{parameters} \leftarrow \text{training data}, \qquad \text{hyperparameters} \leftarrow \text{validation data}, \qquad \text{performance} \leftarrow \text{test data (once)} \tag{2.1}$$

A **parameter** (weight, bias) is learned by the optimiser. A **hyperparameter** (learning rate η, width h, depth L, regularisation λ, batch size) is chosen by the engineer and cannot be learned by gradient descent on the training loss, because the training loss always prefers more capacity.

Step 1 — Lock the test set away

Split the data before anything else, and do not compute a single statistic on the test part until the very end.

$$\mathcal{D} \;\rightarrow\; \mathcal{D}_{train} + \mathcal{D}_{test} \quad \text{(80/20 or 90/10)} \tag{2.2}$$

Justification: every look at test performance is information, and any decision made using that information makes the test set part of model selection. It then no longer measures generalisation; it measures how well the model was fitted to the test set, which is optimistically biased. The bias grows with the number of models compared — selecting the best of 50 models on the test set is a 50-way fit.

Step 2 — Split the training pool into train and validation

$$\mathcal{D}_{train} \;\rightarrow\; \mathcal{D}_{fit} + \mathcal{D}_{val} \quad \text{(80/20)} \tag{2.3}$$

Parameters are fitted on $\mathcal{D}_{fit}$; hyperparameter configurations are ranked on $\mathcal{D}_{val}$.

Justification: $\mathcal{D}_{val}$ plays the role of unseen data during development, so a decision taken on it stays honest for the final test evaluation. Using a separate validation set from a *different source* (e.g.\ another year or site) is even better, because it also exposes distribution shift.

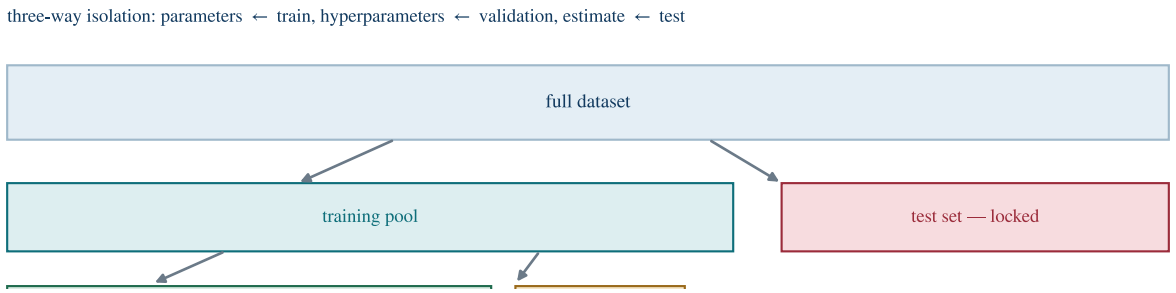


Fig. 2.1 Three-way isolation of the data: parameters from the training pool, hyperparameters from validation, final estimate from the untouched test set.

Step 3 — Choose the search space in a justified way

Hyperparameter	Candidate grid	Justification
Learning rate η	$\{10^{-1}, 3\times10^{-2}, 10^{-2}, 3\times10^{-3}, 10^{-3}, 10^{-4}\}$	logarithmic: the useful range spans orders of magnitude (Question 5, Unit I)
Hidden size h	$\{32, 64, 128, 256, 512\}$	powers of two, again logarithmic in effect
Depth L	$\{1, 2, 3, 4\}$	small integers; depth beyond 4 rarely helps on tabular data
Batch size B	$\{32, 64, 128\}$	hardware efficiency versus gradient noise (Question 3)
Regularisation λ	$\{0, 10^{-5}, 10^{-4}, 10^{-3}\}$	log scale; only after the three main ones are fixed

Justification for the ordering: the learning rate dominates; a model with a bad η looks bad whatever its size. Therefore tune η first with a modest architecture, then width, then depth, then regularisation — a coordinate search instead of a full grid. A full grid costs

$$N_{configs} = |\eta| \times |h| \times |L| = 6 \times 5 \times 4 = 120 \text{ training runs} \tag{2.4}$$

which is why the coordinate order matters in practice.

Step 4 — Use k-fold cross-validation when data is scarce

If the training pool is small, a single 80/20 split wastes data and gives a noisy estimate. Use $k = 5$ (or 10) folds:

$$\mathrm{CV}(\theta) = \frac{1}{k}\sum_{i=1}^{k} \mathrm{Err}\left(f^{(-\mathcal{F}_i)}, \mathcal{F}_i\right) \tag{2.5}$$

Each configuration is trained k times, each time holding out a different fold; the average and standard deviation over folds give a mean and an uncertainty for the configuration.

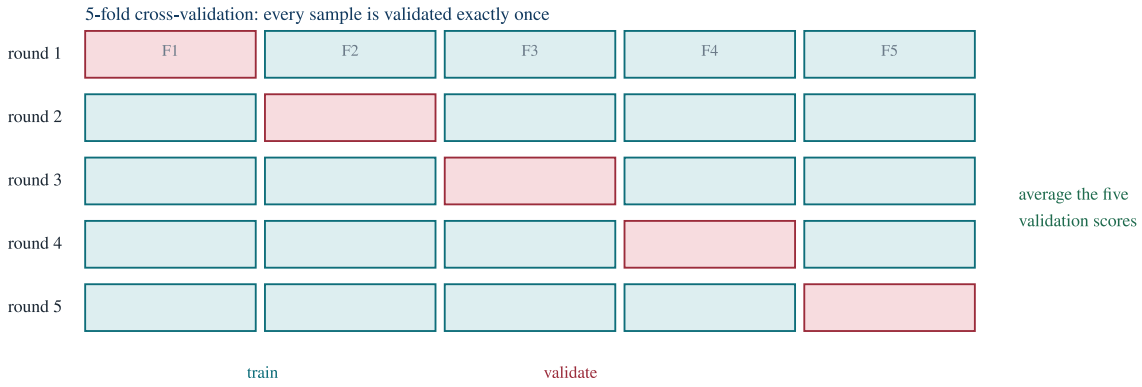


Fig. 2.2 k-fold cross-validation: every sample is used for training $k-1$ times and for validation exactly once.

Justification: all samples contribute to validation exactly once, so the estimate uses the whole pool, and the fold-to-fold standard deviation quantifies how sensitive the choice is. The cost is a factor k in compute.

Step 5 — Fit every preprocessing step inside the fold

The mean, the standard deviation, the PCA basis and any imputation must be fitted on the training part of each fold and merely *applied* to the validation part.

$$\mu_{fold} = \mathrm{mean}(\mathcal{D}_{fit}^{(fold)}), \qquad z = \frac{x - \mu_{fold}}{\sigma_{fold}} \tag{2.6}$$

Justification: computing the scaler on the full dataset leaks the validation distribution (its mean and variance) into training, which produces an optimistic estimate — a subtle but real form of contamination, even though the labels are never seen.

Step 6 — Select, then freeze, then retrain

1. Rank all configurations by mean validation score; select $\theta^{*} = \arg\min_{\theta} \mathrm{CV}(\theta)$.
2. **Freeze** η, h, L and λ. No further changes may be driven by any later measurement.
3. Optionally retrain the selected configuration on the whole training pool ($\mathcal{D}_{fit} \cup \mathcal{D}_{val}$) for the same number of epochs found best — this uses the extra 20\% of data. The retrained model is the final one.
4. With **nested cross-validation**, the inner loop selects hyperparameters and the outer loop estimates performance, which removes the optimism of having selected on the same folds.

Step 7 — Evaluate once on the test set

$$\text{Test error} = \mathrm{Err}(f_{\theta^{*}}, \mathcal{D}_{test}) \tag{2.7}$$

Report it with its confidence interval, and report the validation score alongside. If the test score is much worse than the validation score, the likely cause is a distribution difference between the splits, not a bug in the model.

Strategy	Data used for validation	Bias of the estimate	Variance of the estimate	When to use
Hold-out 80/20	20\% only	moderate (small val set)	high	large datasets, quick iteration
k-fold ($k = 5$)	all data, once each	low	moderate	standard choice
Leave-one-out	$m - 1$ per fold	lowest	high, very expensive	tiny datasets
Stratified k-fold	class-balanced folds	low	lower	imbalanced classes
Time-series split	past only	low	moderate	temporal data (no future leakage)
Nested CV	inner selects, outer estimates	lowest	moderate | expensive	small data + many hyperparameters

CONTAMINATION CHECKLIST

(i) Never tune on the test set. (ii) Never fit scalers/PCA on validation or test data. (iii) Never use test performance to decide when to stop training. (iv) Never report the best of several test evaluations as "the" result. (v) Never shuffle time-series data before splitting.

GOLDEN LINE

Parameters learn from training data, hyperparameters are chosen on validation data, and the test set is opened exactly once. Every preprocessing statistic must be fitted inside the training fold to avoid silent leakage.

KEYWORDS TO WRITE IN THE EXAM

Validation set · hyperparameter · test-set contamination · hold-out split · k-fold cross-validation · stratification · nested CV · data leakage · model selection · generalisation estimate

Q3 Compare batch gradient descent and stochastic gradient descent for training a large deep network. Evaluate their computational behaviour, gradient noise and practical suitability.

CO2 · L5 (Design) · 10 Marks

The two estimators of the same gradient

The loss is an average over the m training samples:

$$J(\theta) = \frac{1}{m}\sum_{i=1}^{m} \ell\big(f(x^{(i)};\theta), y^{(i)}\big) \tag{3.1}$$

Both algorithms perform the same update rule $\theta \leftarrow \theta - \eta\hat{g}$; they differ only in how the gradient estimate $\hat{g}$ is formed.

$$\text{batch: } \hat{g} = \frac{1}{m}\sum_{i=1}^{m}\nabla\ell_i \qquad \text{SGD: } \hat{g} = \nabla\ell_i,\ \ i \sim \text{Uniform}\{1,\dots,m\} \tag{3.2}$$

A mini-batch of size B interpolates between them: $\hat{g} = 1/B\Sigma_{i\in\mathcal{B}}\nabla\ell_i$.

Unbiasedness and noise

Both estimators are unbiased, because the expectation of a sample average is the average:

$$\mathbb{E}[\hat{g}_B] = \nabla J(\theta) \quad \forall B \tag{3.3}$$

The variance of the estimate, however, depends on the batch size:

$$\mathrm{Var}(\hat{g}_B) = \frac{\sigma^2}{B}, \qquad \sigma^2 = \mathrm{Var}(\nabla \ell_i) \tag{3.4}$$

So SGD ($B = 1$) has the largest gradient noise, batch GD ($B = m$) has none, and the standard deviation shrinks as $1/\sqrt{B}$. Halving the noise costs four times the batch size — that square root is the key economic fact.

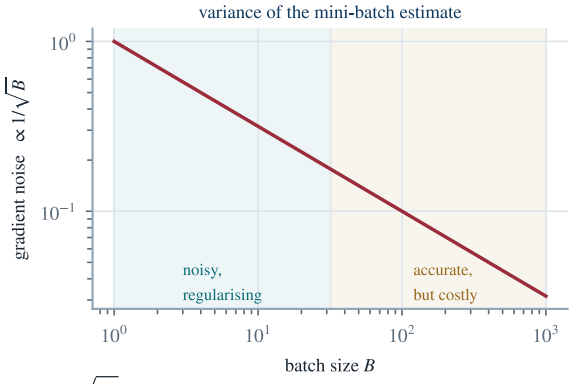


Fig. 3.1 Gradient noise versus batch size: the $1/\sqrt{B}$ law. Small batches are noisy but cheap; large batches are accurate but expensive per step.

Computational behaviour

Aspect	Batch gradient descent	Stochastic / mini-batch GD
Gradient per step	exact, uses all m samples	one sample (or B), noisy
Cost per step	$O(m)$ forward + backward passes	$O(B) \ll O(m)$
Updates per epoch	1	m/B (e.g.\ 500 for $m = 50{,}000$, $B = 100$)
Memory	must hold all activations for all m	only B samples at a time
Hardware use	good vectorisation, but often too large for GPU memory	ideal for GPU/SIMD parallelism
Convergence (smooth convex)	$O(1/k)$, monotone decrease	$O(1/\sqrt{k})$ for the last iterate, but $O(1/k)$ for the averaged iterate
Escape from saddle points	slow — noise-free dynamics stall at $\nabla J = 0$	noise helps escape shallow saddles and poor local minima
Wall-clock to a given loss	usually far slower for large m	usually far faster, despite more iterations

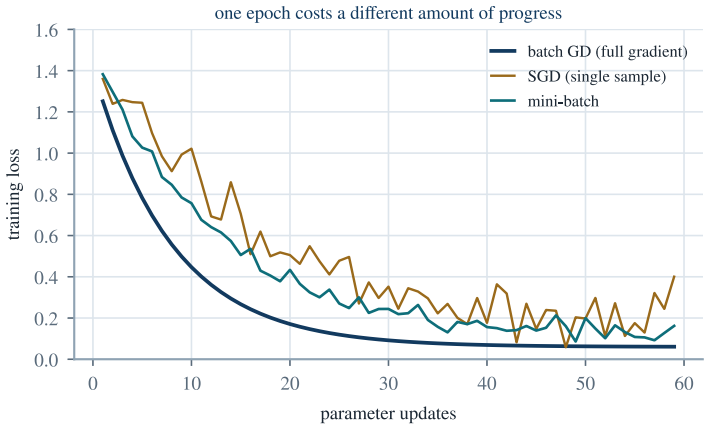


Fig. 3.2 Trace of the training loss against the number of parameter updates. Batch GD moves smoothly; SGD jitters but travels much further per unit of computation.

Why the noise is not purely harmful

1. **Exploration.** At a saddle point $\nabla J = 0$ the batch gradient vanishes and progress stops. The stochastic gradient does not vanish there, so SGD continues and can leave flat regions.

2. **Implicit regularisation.** The noise discourages the solution from settling into a sharp, high-curvature minimum, which tends to generalise better.
3. **Faster early progress.** The loss surface is steep far from the optimum, and a noisy estimate is good enough; the expensive exact gradient buys accuracy where it is not yet needed.

The convergence trade-off, stated properly

For an L-smooth convex objective, batch GD with $\eta = 1/L$ satisfies

$$J(\theta_k) - J^* \le \frac{L\|\theta_0 - \theta^*\|^2}{2k} = O\left(\frac{1}{k}\right) \tag{3.5}$$

whereas SGD with a decaying step $\eta_k \propto 1/\sqrt{k}$ satisfies only

$$\mathbb{E}[J(\theta_k)] - J^* = O\left(\frac{1}{\sqrt{k}}\right) \tag{3.6}$$

But the comparison is unfair in iterations, not time. One batch step costs m gradient evaluations, so per unit of computation the two are comparable (both $O(1/\varepsilon)$), while SGD typically reaches a useful loss far earlier in wall-clock time on large datasets. The noise floor also matters: with a constant η, SGD converges only to a neighbourhood of radius $O(\eta\sigma^2)$, which is why η must decay.

Practical design rules

1. **Batch size.** Use the largest B that fits memory and still gives enough updates per epoch; $B = 32$–256 is the usual range. Bigger batches need a proportionally larger η (**linear scaling rule**: $\eta \propto B$), with a warm-up for the first few hundred steps.
2. **Epochs.** One epoch $= \lceil m/B \rceil$ updates. Report progress in epochs, not iterations, so runs are comparable.
3. **Shuffling.** Reshuffle every epoch; without it the sample order becomes an accidental curriculum and the gradient is biased in blocks.
4. **Learning-rate schedule.** Start moderately, decay (step, cosine or $1/t$). Momentum or Adam stabilises the noisy direction.
5. **Large-batch training.** Very large batches reduce noise so much that generalisation can degrade; remedies are longer warm-up, higher η, or training longer.

Evaluation and conclusion

For a large deep network the *mathematical* objective is the same, but the *engineering* verdict is clear:

- **Batch GD is impractical** beyond small datasets: one update per epoch wastes the epoch, and the memory needed for all activations and the exact gradient is usually unavailable.
- **Pure SGD** ($B = 1$) is too noisy and wastes vectorised hardware.
- **Mini-batch SGD is the practical optimum**: it approximates the full gradient well enough ($\mathrm{Var} \propto 1/B$), exploits GPU parallelism, needs bounded memory, provides m/B updates per epoch, and its noise acts as a useful regulariser and saddle-escape mechanism.

> **MAKE THE EVALUATION EXPLICIT**
> The examiner wants: same update rule, different gradient estimator; unbiasedness; $\mathrm{Var} \propto 1/B$; cost per step $O(B)$ versus $O(m)$; m/B updates per epoch; and the wall-clock verdict. Quote the $1/\sqrt{B}$ noise law — it is the one formula that shows real understanding.

> **GOLDEN LINE**
> Batch GD uses the exact gradient once per epoch; SGD uses a noisy estimate many times per epoch. The noise variance falls as $1/B$, and for large networks mini-batch SGD wins on computation, memory and generalisation.

KEYWORDS TO WRITE IN THE EXAM
Batch gradient descent · stochastic gradient descent · mini-batch · unbiased estimator · gradient noise $\propto 1/\sqrt{B}$ · updates per epoch · convergence rate · saddle point · linear scaling rule · learning-rate schedule

Q4 **A network learns the training examples but fails on unseen examples. Evaluate whether changing the architecture, collecting more data, or changing the learning algorithm is the most appropriate first intervention, and justify your choice.**

CO2 · L5 (Design) · 10 Marks

What the symptom already tells us

"Learns the training examples but fails on unseen examples" is the definition of **overfitting**: $E_{train} \ll E_{val}$ with a large gap (Question 1). The model has sufficient capacity — the training error proves it — so the deficiency is not representational but statistical: the network has fitted sampling noise that does not recur in new data.

$$E_{train} \approx 0, \qquad E_{val} \gg E_{train} \quad \Longrightarrow \quad \text{high variance, low bias} \tag{4.1}$$

That single inference already rules out one of the three options: **changing the architecture is the wrong first move**, because architecture changes are aimed at *capacity*, and capacity is not the missing ingredient.

Step 0 — confirm it is really overfitting

Before intervening, eliminate the three impostors that imitate the symptom:

- **Leakage.** If validation features were built using target information, a "poor" validation score may actually be a broken protocol, not overfitting.
- **Distribution shift.** If validation data come from a different source, the model may be correct but misapplied; the fix is then data or domain adaptation, not regularisation.
- **Label noise / duplicates.** Duplicated rows across splits inflate E_{train} optimism; near-duplicate validation rows hide the true gap.

Only if these are excluded does the diagnosis stand.

Evaluating the three candidate interventions

Intervention	What it changes	Effect on E_{train}	Effect on E_{val}	Cost	Verdict for this symptom
More / better data	the statistical problem itself	rises slightly	falls, gap closes	data collection effort	**first choice** — attacks the cause
Architecture change	capacity, inductive bias	falls	usually rises or unchanged	engineering time	wrong direction here
Learning-algorithm change	optimisation path, implicit regularisation	similar	small, unreliable change	tuning time	secondary

1. **Collecting (or augmenting) data** attacks the cause directly. Variance is the sensitivity of the fit to the particular sample; a larger, more varied sample averages that sensitivity out. The learning-curve test makes this quantitative: if E_{val} is still falling steeply as m grows, more data will help.

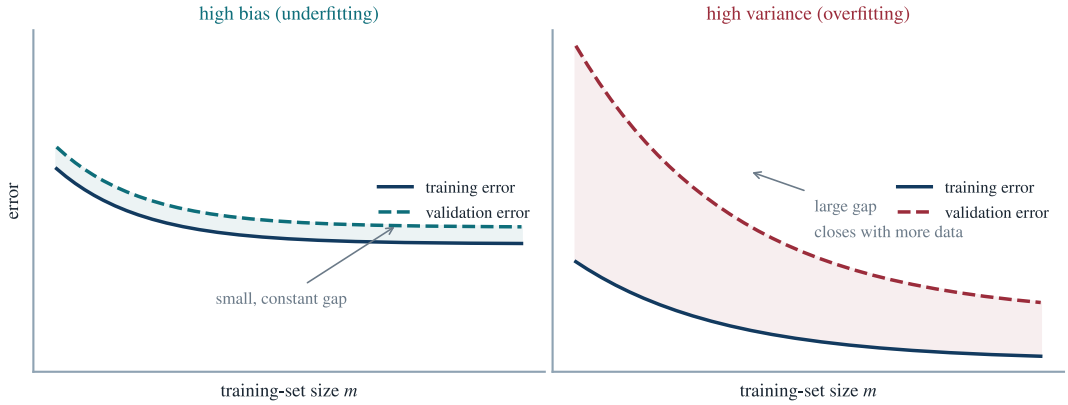


Fig. 4.1 The right-hand panel is this question: a large gap that narrows as training data grows. The left-hand panel would be underfitting, where more data would not help.

1. **Changing the architecture** would be correct only if E_{train} were also high (high bias). A bigger model makes the gap worse; a smaller model can help, but it does so by *reducing capacity*, which is a blunt instrument compared with regularisation and often increases bias unnecessarily.
2. **Changing the learning algorithm** (optimiser, learning rate, initialisation) mostly changes how well the training loss is minimised. Since the training loss is already low, the room for improvement is small; the optimiser is not the bottleneck.

The practical decision order

The three options are not equally cheap, so the justified order is *cheapest effective first*:

1. **1 — Regularise before collecting.** Weight decay, dropout, early stopping and data augmentation are free or nearly so, and they reduce variance immediately. If they close the gap, the data-collection cost is avoided.
2. **2 — Augment before labelling.** Synthetic variation (crops, flips, noise, mixtures) increases effective sample size at almost no cost when it respects the semantics of the task.
3. **3 — Then collect more real data.** The most reliable fix, but the most expensive; justify it by showing that the learning curve is still descending.
4. **4 — Only then reduce capacity** (fewer layers/units), and only if the gap persists.
5. **5 — Changing the learning algorithm** is the last lever, useful mainly when the model is unstable or the loss is not converging at all.

Answer to the "which is most appropriate" question

For a network that already fits the training data:

$$\text{most appropriate first intervention} = \text{increase the effective amount of data} \tag{4.2}$$

(either by **collecting** more examples or, more cheaply, by **augmenting** them and adding **regularisation** which has the same mathematical effect of reducing variance). Changing the architecture is *not* appropriate first, because it targets capacity, and the capacity is demonstrably sufficient; changing the learning algorithm is *not* appropriate first, because optimisation is demonstrably working — the training loss is low.

Justification in bias–variance terms

The regularised objective makes the mechanism explicit:

$$\min_{\theta} \frac{1}{m} \sum_{i=1}^{m} \ell\big(f(x^{(i)};\theta), y^{(i)}\big) + \lambda \|\theta\|^2 \tag{4.3}$$

Increasing m or λ both shrink the effective weight magnitude, which lowers variance at the cost of a small increase in bias. Since this model sits on the low-bias side of the trade-off, paying a little bias is a good trade; adding parameters would move in exactly the wrong direction.

THE TRAP IN THIS QUESTION

Many answers say "the model overfits, so we need a better architecture". That is precisely wrong: a stronger architecture increases capacity and therefore increases variance. The marks are for recognising that the observed training error *rules out* the capacity explanation.

GOLDEN LINE

A low training error with a high validation error is high variance. The first intervention must reduce variance — more data, augmentation and regularisation — not a new architecture (capacity is already sufficient) and not a different optimiser (optimisation is already succeeding).

KEYWORDS TO WRITE IN THE EXAM

Overfitting · generalisation gap · high variance · capacity · data augmentation · regularisation · weight decay · early stopping · learning curve · bias–variance trade-off · distribution shift

Q5 Analyse the relationship among maximum likelihood, output-unit choice and loss-function choice for regression and binary classification in feedforward networks.

CO2 · L4 (Analyse) · 10 Marks

One principle, many losses

Every standard loss function in deep learning is the **negative log-likelihood** of a probabilistic model of the target. Maximum likelihood therefore supplies a single derivation rule that produces the squared error for regression and the cross-entropy for classification, and it simultaneously dictates which output activation must be used.

$$\theta_{ML} = \arg\max_{\theta} \; p(\mathcal{D} \mid \theta) = \arg\max_{\theta} \prod_{i=1}^{m} p\left(y^{(i)} \mid x^{(i)}; \theta\right) \tag{5.1}$$

Taking logarithms (monotone, and numerically safer — Unit I, Question 3) turns the product into a sum:

$$\theta_{ML} = \arg\min_{\theta} \sum_{i=1}^{m} -\log p\left(y^{(i)} \mid x^{(i)}; \theta\right) \tag{5.2}$$

The quantity being minimised **is** the loss. Hence: choose a distribution, and the loss follows automatically. Maximum likelihood is also the estimator that is consistent and asymptotically efficient under correct model specification.

Regression: Gaussian likelihood gives squared error

Assume $y = x^{\top} w + \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, \sigma^2)$, so that

$$p(y \mid x; \theta) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y - \hat{y})^2}{2\sigma^2}\right) \tag{5.3}$$

Taking the negative log:

$$-\log p(y \mid x; \theta) = \frac{(y - \hat{y})^2}{2\sigma^2} + \frac{1}{2}\log(2\pi\sigma^2) \tag{5.4}$$

The second term does not depend on θ. Therefore

$$\arg\min_{\theta} \sum_{i} -\log p\left(y^{(i)} \mid x^{(i)}; \theta\right) = \arg\min_{\theta} \sum_{i} \left(y^{(i)} - \hat{y}^{(i)}\right)^2 \tag{5.5}$$

MSE is maximum likelihood under Gaussian noise. The variance σ^2 only scales the loss; the minimiser is unchanged. The **output unit must therefore be the identity** ($\hat{y} = z$), because the Gaussian's mean parameter is unconstrained. Using a

sigmoid on a regression output would restrict $\hat{y}$ to $(0, 1)$ and is a modelling error, not merely a slow start.

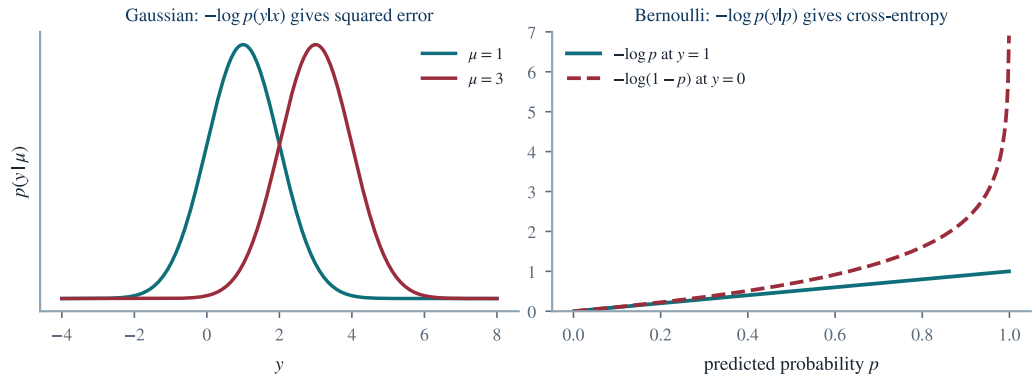


Fig. 5.1 Left: Gaussian likelihood gives the squared-error loss; its mean parameter is unconstrained, so the output unit is linear. Right: Bernoulli likelihood gives the binary cross-entropy; its parameter is a probability, so the output unit is a sigmoid.

Binary classification: Bernoulli likelihood gives cross-entropy

For $y \in \{0, 1\}$ with $p(y = 1 \mid x) = \hat{p} = \sigma(z)$, the Bernoulli likelihood of one example is

$$p(y \mid x) = \hat{p}^{\,y}(1 - \hat{p})^{1-y} \tag{5.6}$$

and its negative log is the **binary cross-entropy**:

$$-\log p(y \mid x) = -[y \log \hat{p} + (1 - y)\log(1 - \hat{p})] \tag{5.7}$$

The output unit must be a sigmoid, because the Bernoulli parameter is a probability in $(0, 1)$ and the sigmoid is exactly the map from the real line to that interval. The two choices are not independent: the *likelihood* fixes the loss, and the *parameter space* fixes the output unit.

Two equivalent ways to see the same loss:

- **Information view:** $\mathrm{CE}(p, q) = H(p) + \mathrm{KL}(p \| q)$; minimising cross-entropy is minimising the KL divergence to the empirical distribution, since $H(p)$ is constant.
- **Gradient view:** with $\hat{p} = \sigma(z)$ and BCE loss, the chain rule collapses beautifully to

$$\frac{\partial L}{\partial z} = \hat{p} - y \tag{5.8}$$

The factor $\sigma'(z) = \hat{p}(1 - \hat{p})$ cancels the $1/\hat{p}$ and $1/(1 - \hat{p})$ of the loss denominators. This is **why sigmoid + BCE must be implemented as a single fused operation**: computing them separately reintroduces the vanishing factor and the numerical instability.

Why the pairing matters (the analysis)

Task	Assumed distribution	Output unit	Loss	Gradient w.r.t. logit
Regression	$\mathcal{N}(\hat{y}, \sigma^2)$	identity	$1/2(y - \hat{y})^2$	$\hat{y} - y$
Binary classification	Bernoulli($\hat{p}$)	sigmoid	binary cross-entropy (Eq. 5.7)	$\hat{p} - y$
Multi-class	Categorical($\hat{p}_k$)	softmax	categorical cross-entropy	$\hat{p}_k - y_k$
Count / Poisson	Poisson(λ)	exponential	$\lambda - y \log \lambda$	$1 - y/\lambda$

The last column is the striking result: **for the matched (canonical) pairing the gradient with respect to the pre-activation is always "prediction minus target".** This is a general property of exponential-family models: the natural (canonical) link makes the Hessian positive semi-definite and the optimisation well conditioned.

What goes wrong when the pairing is mismatched

- **MSE with a sigmoid** (a "matched" loss but the wrong one): the gradient acquires the extra factor $\hat{p}(1-\hat{p})$, so it vanishes exactly when the model is confidently wrong. Learning stalls, and the loss is not convex in the logit.
- **BCE with a linear output** produces log of a value outside $(0, 1)$ — a domain error, or NaN after clamping.
- **Softmax with MSE** on one-hot targets loses the probabilistic interpretation and reduces the effective gradient by an extra factor.
- **Sigmoid + BCE computed separately** overflows/underflows; use *BCEWithLogits* / *log_softmax + NLLLoss* fused forms.

Derivation of the multi-class case (link to Question 6)

With categorical targets $y_k \in \{0, 1\}$, $\Sigma_k y_k = 1$ and $\hat{p}_k = \text{softmax}(z)_k$:

$$-\log p(y \mid x) = -\sum_{k=1}^{K} y_k \log \hat{p}_k \quad \Longrightarrow \quad \frac{\partial L}{\partial z_k} = \hat{p}_k - y_k \tag{5.9}$$

For a one-hot target this is simply $\hat{p}_k$ for the wrong classes and $\hat{p}_k - 1$ for the correct one — an intuitive "push down the wrong, push up the right" update.

Summary of the relationship

1. **Maximum likelihood** is the single principle: minimise the negative log-likelihood of the observed targets.
2. **The assumed distribution** determines the loss: Gaussian → squared error, Bernoulli → binary cross-entropy, categorical → cross-entropy.
3. **The parameter domain of that distribution** determines the output unit: real line → identity, probability → sigmoid, simplex → softmax.
4. Choosing them jointly yields the clean gradient $\hat{y} - y$, convexity in the logits for the exponential-family cases, and numerical stability; choosing them independently breaks one of the three.

ONE SENTENCE THAT CARRIES THE ANSWER

"Maximum likelihood turns a choice of noise model into a choice of loss: Gaussian noise gives squared error with a linear output, Bernoulli noise gives binary cross-entropy with a sigmoid, and categorical noise gives cross-entropy with a softmax — and in each matched case the gradient with respect to the logit is simply prediction minus target."

GOLDEN LINE

Pick the distribution, and the loss and output unit follow from maximum likelihood. Gaussian gives MSE with a linear unit; Bernoulli gives BCE with a sigmoid; categorical gives CE with a softmax.

KEYWORDS TO WRITE IN THE EXAM

Maximum likelihood · negative log-likelihood · Gaussian likelihood · squared error · Bernoulli · binary cross-entropy · softmax · categorical cross-entropy · output unit · canonical link · fused loss · KL divergence

Q6 Design a deep feedforward architecture for multiclass classification and justify the hidden layers, hidden-unit nonlinearity, output layer and objective function.

CO2 · L5 (Design) · 10 Marks

Problem specification

Input $x \in \mathbb{R}^d$, target $y \in \{1, \ldots, K\}$ with $K > 2$ classes and no ordering among them. The design must produce, for every input, a probability vector over the K classes, and it must be trainable by gradient descent.

Architecture

A deep feedforward network with L hidden layers:

$$h^{(1)} = \sigma(W^{(1)}x + b^{(1)}), \quad h^{(l)} = \sigma(W^{(l)}h^{(l-1)} + b^{(l)}), \quad z = W^{(L+1)}h^{(L)} + b^{(L+1)} \tag{6.1}$$

$$\hat{p} = \mathrm{softmax}(z), \qquad \hat{p}_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad \hat{y} = \arg\max_k \hat{p}_k \tag{6.2}$$

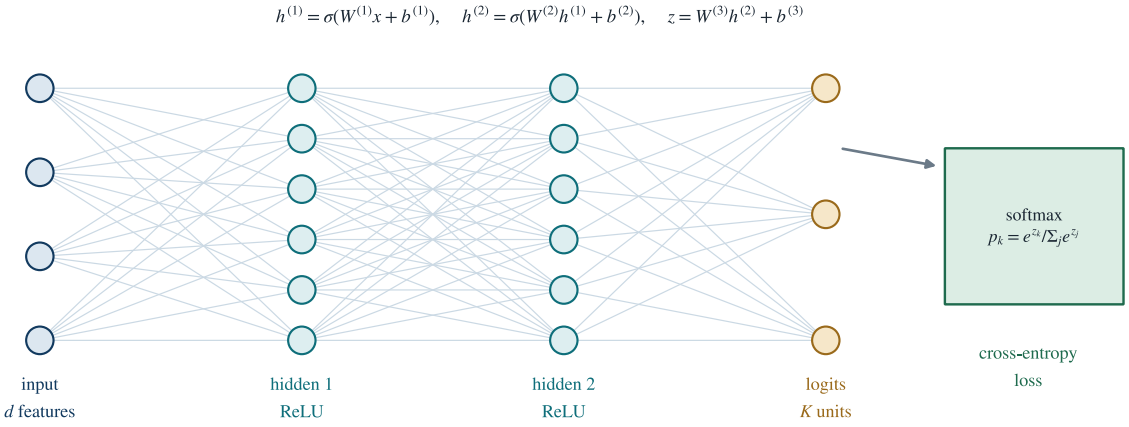


Fig. 6.1 A four-stage design: input, two ReLU hidden layers, a linear layer producing K logits, and a softmax that converts logits into a probability vector.

- ***Parameter count per layer***

$$n_{params}^{(l)} = (n_{l-1} + 1)n_l \tag{6.3}$$

For example $784 \to 256 \to 128 \to 10$ on MNIST: $(784 + 1) \cdot 256 = 200{,}960$, then $257 \cdot 128 = 32{,}896$, then $129 \cdot 10 = 1{,}290$, giving 235,146 parameters in total.

Justification of each design decision

Choice	What was chosen	Justification
Hidden layers (L)	2–3	enough depth to compose nonlinear features; deeper only with residual connections and more data
Hidden width (h)	128–512, decreasing	capacity per layer; decreasing widths compress the representation towards the classifier
Hidden activation	ReLU $\max(0, z)$	non-saturating for $z > 0$, so gradients do not vanish; sparse; cheap; avoids the vanishing-gradient problem of sigmoid/tanh
Output layer	K linear units (logits)	one score per class, unconstrained reals — exactly the domain the categorical model needs
Output activation	softmax	maps $\mathbb{R}^K$ onto the probability simplex: $\hat{p}_k \geq 0$, $\Sigma_k \hat{p}_k = 1$; shift-invariant and stable with max-shift
Objective	categorical cross-entropy	negative log-likelihood of the categorical distribution (Question 5); convex in the logits
Target encoding	one-hot vectors	matches the categorical distribution; avoids implying an order between classes
Prediction rule	$\arg\max_k \hat{p}_k$	Bayes-optimal decision under 0–1 loss

Choice	What was chosen	Justification
Initialisation	He/Xavier	keeps activation variance stable through depth

- ***Why ReLU rather than sigmoid***

$$\sigma'(z) = \sigma(z)(1-\sigma(z)) \le 0.25 \;\Rightarrow\; \text{gradients shrink by } \ge 4\times \text{ per layer} \tag{6.4}$$

ReLU has derivative 1 for $z > 0$ and 0 otherwise, so the gradient of a deep stack does not decay geometrically. Its piecewise-linearity also means the network's output is piecewise linear in the input, which is why deep ReLU networks can represent highly folded decision boundaries.

- ***Why softmax rather than K independent sigmoids***

Sigmoids assume K independent binary decisions. For mutually exclusive classes we require $\Sigma_k p_k = 1$; softmax enforces this exactly and couples the scores, so raising one class's score necessarily lowers the others.

$$\frac{\partial \hat{p}_k}{\partial z_j} = \hat{p}_k\left(\delta_{kj} - \hat{p}_j\right) \tag{6.5}$$

Training objective

$$J(\theta) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{K} y_k^{(i)} \log \hat{p}_k^{(i)} + \frac{\lambda}{2}\|\theta\|^2 \tag{6.6}$$

The first term is the mean negative log-likelihood; the second is optional ℓ_2 regularisation (Unit I, Question 6). Because the loss is the NLL of a categorical distribution, the gradient with respect to the logits is the clean difference

$$\frac{\partial J}{\partial z_k} = \hat{p}_k - y_k \tag{6.7}$$

and back-propagation then continues into $W^{(L+1)}$ and below by the chain rule. In practice the loss is computed as *cross_entropy(logits, targets)*, which fuses log-softmax and NLL to avoid overflow.

Design procedure

1. **Standardise the inputs** and encode labels one-hot; split train/validation/test (Unit II, Question 2).
2. **Fix the output block**: K units + softmax + cross-entropy — this is not a free choice, it follows from the task.
3. **Choose depth and width** from the data size: start shallow ($L = 1$–2, $h \approx \sqrt{d \cdot K}$), then grow while validation error improves (Question 7).
4. **Use ReLU** in hidden layers, He initialisation, and optionally dropout/batch-norm.
5. **Train with mini-batch SGD/Adam**, tune η on validation data, monitor both cross-entropy and accuracy.
6. **Evaluate** with a confusion matrix and per-class precision/recall or macro-F1, not accuracy alone — accuracy hides class imbalance.
7. **Diagnose**: high training loss → underfitting (more capacity); low training loss with high validation loss → overfitting (regularise).

Worked example — design for a 10-class problem with $d = 100$, $m = 20{,}000$

- Architecture: $100 \to 256 \to 128 \to 10 = (101)(256) + (257)(128) + (129)(10) = 59{,}754$ parameters — about 3 parameters per training sample, a reasonable ratio.
- Output: 10 logits, softmax, one-hot targets, categorical cross-entropy.
- Training: Adam, $\eta = 10^{-3}$, batch 128, $\ell_2 = 10^{-4}$, early stopping on validation cross-entropy.
- Expected behaviour: training cross-entropy falls to ≈ 0.05; if validation cross-entropy rises after epoch 12, overfitting has begun and early stopping returns the epoch-12 weights.

DESIGN QUESTIONS WANT REASONS, NOT A LIST

For each block, write *what it is* and *why the task forces it*: softmax because the classes are mutually exclusive; cross-entropy because the likelihood is categorical; ReLU because gradients survive depth; K output units because there are K classes. That "because" is where the marks are.

GOLDEN LINE

Input → ReLU hidden layers → K logits → softmax → categorical cross-entropy, with one-hot targets and arg max prediction. Each choice is forced by the categorical likelihood and by the need for gradients that survive depth.

KEYWORDS TO WRITE IN THE EXAM

Multiclass classification · softmax · logits · one-hot encoding · categorical cross-entropy · ReLU · vanishing gradient · He initialisation · parameter count · confusion matrix · macro-F1

Q7 Analyse how depth, width and hidden-unit choice affect representational capacity in a deep feedforward network.

CO2 · L4 (Analyse) · 10 Marks

Definitions

Capacity is the size and richness of the set of functions $\mathcal{H}$ that the architecture can represent. It is not one number; it is controlled by three knobs:

- **Width** h: number of units per hidden layer.
- **Depth** L: number of hidden layers (parameterised transformations composed).
- **Hidden-unit choice**: which activation and how many units per layer — including non-uniform widths.

$$\text{parameters } P = (d+1)h_1 + \sum_{l=2}^{L}(h_{l-1}+1)h_l + (h_L+1)K \tag{7.1}$$

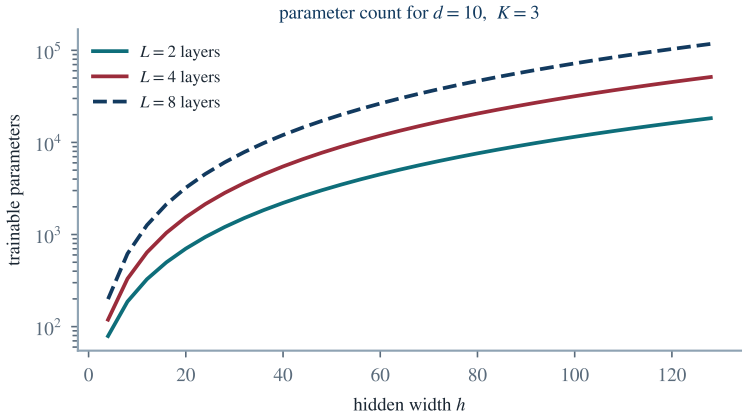


Fig. 7.1 Parameter count against hidden width for three depths ($d = 10$, $K = 3$, logarithmic axis). Every extra layer adds a multiplicative factor at every width, so depth is the cheaper way to grow capacity.

Width and the universal approximation theorem

A network with a single hidden layer of sufficient width and a non-polynomial activation can approximate any continuous function on a compact set to arbitrary accuracy:

$$\forall f \in C(\mathcal{X}),\ \forall \varepsilon > 0\ \ \exists h,\ W:\quad \sup_{x \in \mathcal{X}} \left| f(x) - \sum_{j=1}^{h} w_j \sigma\left(a_j^{\top} x + b_j\right) \right| < \varepsilon \tag{7.2}$$

Interpretation: width is enough, in principle. **But the theorem is not a design recipe** — it says nothing about how many units are needed, whether gradient descent will find them, or how many samples are required. In the worst case the required width grows exponentially with input dimension.

Depth and hierarchical composition

Depth gives something width does not: **composition**. A deep network evaluates

$$f(x) = f_L \circ f_{L-1} \circ \cdots \circ f_1(x) \tag{7.3}$$

so it can build features in stages — edges, then textures, then parts, then objects. The classical result is that some functions representable by a deep network of size poly(d) require exponentially many units in a shallow network. Concretely, a piecewise-linear function with R linear regions needs

$$\text{regions}(L, h) \approx O\left(\left(\frac{h}{h_0}\right)^{L} h^{d}\right) \tag{7.4}$$

so the number of linear pieces — the "folding" of the decision boundary — grows **exponentially with depth** and only polynomially with width. Depth is therefore the more parameter-efficient way to increase capacity.

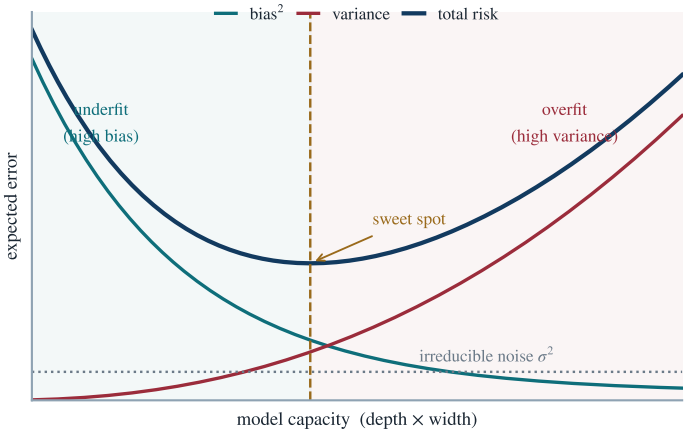


Fig. 7.2 Capacity, whether bought by depth or width, moves the model along the bias–variance trade-off. The optimum is where total risk is minimised, not where capacity is maximised.

The three-way analysis

Property	Increase depth L	Increase width h	Change hidden units / activation
Parameters	grows linearly per added layer	grows quadratically in the layer	depends (wider layer if more units)
Expressivity	exponential gain (composition, linear regions)	polynomial gain	changes smoothness and inductive bias
Feature reuse	strong: later layers reuse earlier features	weak: units act in parallel, not hierarchically	ReLU gives piecewise-linear pieces; tanh gives smooth ones
Optimisation	harder: vanishing/exploding gradients, needs normalisation, residuals, care in initialisation	easier: wider networks are easier to train	saturating units slow learning; ReLU accelerates it
Sample complexity	high — more data needed for the extra flexibility	high	a poor activation can waste capacity entirely
Overfitting risk	high	high	neutral
Practical cost	latency grows with sequential depth	more memory/FLOPs per layer	cheap to change

Effective versus nominal capacity

Nominal capacity (parameter count) is a poor guide on its own. **Effective capacity** is reduced by:

- **Regularisation** — weight decay, dropout, early stopping, augmentation (Unit I, Question 6).
- **Saturation** — with sigmoid/tanh, many units sit in their flat region and contribute little; the usable capacity is far below the nominal one.
- **Dead ReLUs** — units whose pre-activation is negative for all inputs output 0 and can never recover, permanently removing capacity.
- **Data** — capacity is only usable if there are enough samples to estimate the extra parameters. A useful rule of thumb is $P \leq m/10$ for well-conditioned problems, with augmentation relaxing the bound.

Empirically, deep networks generalise even when $P \gg m$ — the "double descent" phenomenon — because the *implicit* regularisation of SGD and the architecture itself restricts which of the many interpolating solutions is found. Capacity theory therefore predicts the *possibility* of overfitting, not its inevitability.

Design guidance

1. **Grow depth before width** when the data has hierarchical structure (images, audio, text), because composition is what such data requires.
2. **Grow width before depth** when the data is tabular and shallow, or when training stability is a concern — wide models are easier to optimise.
3. **Keep the final hidden layer modest** so the classifier head does not dominate the parameter budget; compress progressively.
4. **Match capacity to data**: start small and grow while validation error improves (Question 7 diagnosis), instead of starting large and hoping regularisation saves the model.
5. **Prefer architectural priors over raw capacity**: convolution for images, weight sharing, or residual connections buy expressivity per parameter far more cheaply than adding units.

Worked comparison

Two candidates for a $d = 1{,}000$, $K = 10$ task with $m = 50{,}000$:

- **Wide and shallow:** 1 hidden layer of $2{,}048$ units $\Rightarrow P = (1001)(2048) + (2049)(10) \approx 2.07$ million.
- **Deep and narrow:** 4 hidden layers of 256 units $\Rightarrow P = (1001)(256) + 3(257)(256) + (257)(10) \approx 0.46$ million.

The deep model has $4.5\times$ fewer parameters yet typically represents the task better, because its 4 composed layers create exponentially more linear regions than the single wide layer. The wide model, however, trains more reliably and is preferred when m is small.

THE ANALYSIS THE EXAMINER WANTS

Do not just say "more layers = more capacity". Say *how much more*: linear regions grow as h^L (exponential) but only polynomially in h; parameter count grows linearly in L; and effective capacity is what matters, reduced by regularisation, saturation and dead units.

GOLDEN LINE

Width guarantees approximation; depth guarantees efficiency of approximation. Parameters grow linearly with depth and quadratically with width, while representational power grows exponentially with depth and only polynomially with width.

KEYWORDS TO WRITE IN THE EXAM

Depth · width · hidden units · capacity · universal approximation · hierarchical composition · linear regions · parameter count · vanishing gradient · effective capacity · double descent · overfitting

Q8 **Construct an end-to-end training workflow for a fully connected deep network, starting from dataset partitioning and ending with back-propagation-based parameter updates. Include the key learning decisions that must be fixed before training.**

CO2 · L5 (Design) · 10 Marks

Overview

The workflow has twelve stages. The first six are decisions that must be **fixed before the first gradient step**; the last six form the optimisation loop and its evaluation.

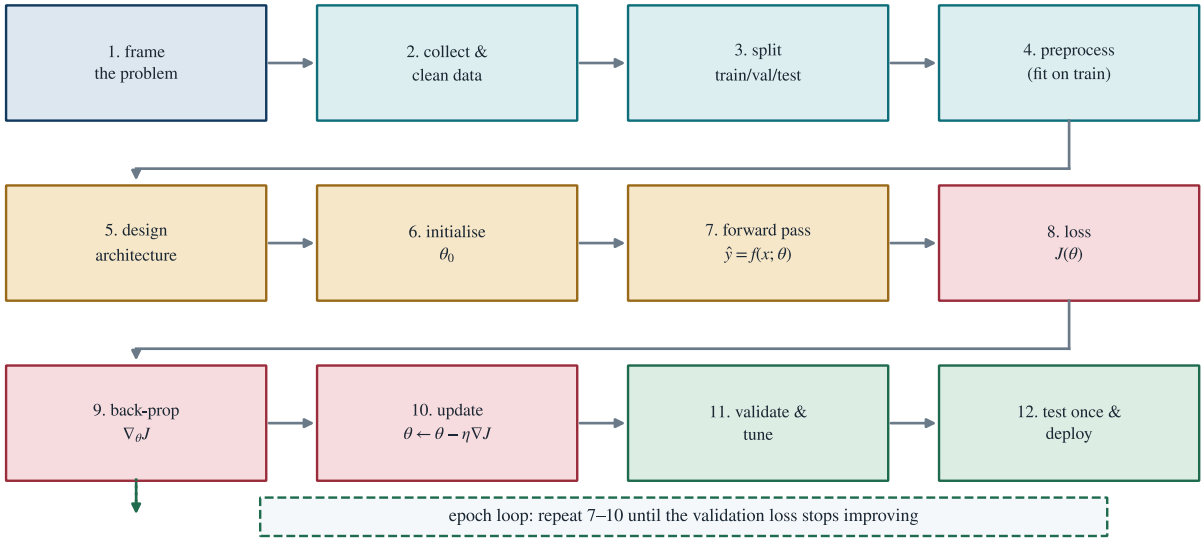


Fig. 8.1 The complete workflow. Stages 1–4 prepare the data, 5–8 define the model and its loss, and 9–12 iterate and evaluate; the dashed arrow is the epoch loop.

Stage 1 — Frame the problem

Fix the input space $\mathcal{X}$, the target space $\mathcal{Y}$ and the loss metric before touching the data. Classification or regression determines the output unit and the loss (Question 5); the metric (accuracy, macro-F1, RMSE) determines what "better" means.

Stage 2 — Collect and clean

Handle missing values, remove duplicates, correct labels, and check class balance. **Duplicates across splits are the most common cause of imaginary accuracy**, so deduplicate before splitting.

Stage 3 — Partition the data

$$\mathcal{D} \rightarrow \mathcal{D}_{train}\ (70\%) + \mathcal{D}_{val}\ (15\%) + \mathcal{D}_{test}\ (15\%) \tag{8.1}$$

Stratify by class, and split *before* any preprocessing so that no statistic crosses the boundary. The test set is opened exactly once, at stage 12.

Stage 4 — Preprocess (fit on training data only)

$$x \leftarrow \frac{x - \mu_{train}}{\sigma_{train}} \tag{8.2}$$

Then optional PCA or feature selection, with every parameter of the transform estimated on $\mathcal{D}_{train}$ and merely applied elsewhere. **Decision to fix now:** whether to standardise, and whether to reduce dimension (Unit I, Question 8).

$$\Sigma_{train} = \frac{1}{m-1} X_{train}^{\top} X_{train}, \qquad Z = XW_k \tag{8.3}$$

Stage 5 — Fix the architecture

Choose depth L, widths h_l, activation σ, and the output layer. Fix the parameter count in advance:

$$P = (d+1)h_1 + \sum_{l=2}^{L}(h_{l-1}+1)h_l + (h_L+1)K \tag{8.4}$$

Decisions to fix: depth, width, activation (ReLU), output unit (softmax or identity), and the parameter budget relative to m (Question 7).

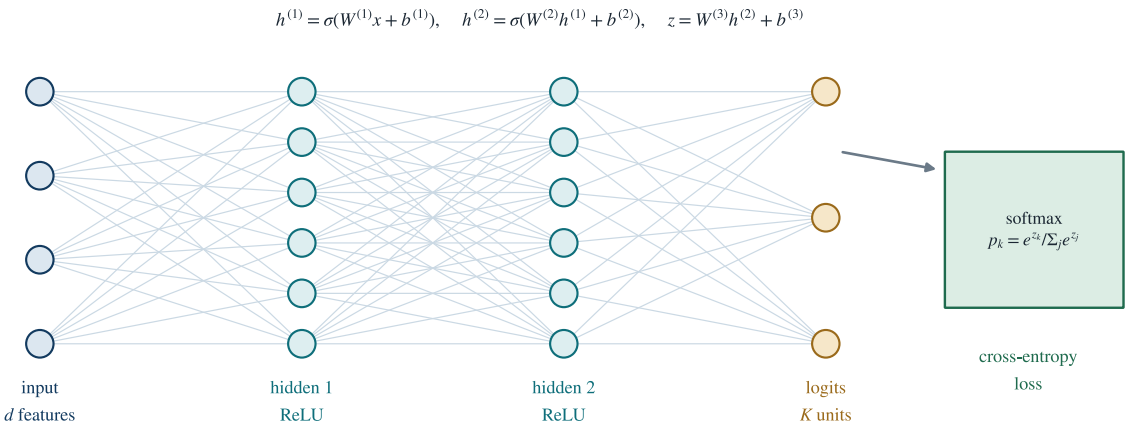


Fig. 8.2 The reference architecture used throughout: ReLU hidden layers, a linear layer producing logits, then softmax and cross-entropy.

Stage 6 — Fix initialisation and regularisation

$$W \sim \mathcal{N}\left(0, \frac{2}{n_{in}}\right) \text{ (He)}, \qquad b = 0 \tag{8.5}$$

Decisions to fix: initialiser (He for ReLU, Xavier for tanh), weight decay λ, dropout rate, and whether to use batch normalisation. These must be chosen before training because they change the optimisation landscape itself.

Stage 7 — Forward pass

For each mini-batch, compute activations layer by layer and cache them (they are needed for back-propagation):

$$a^{(0)} = x, \qquad z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \qquad a^{(l)} = \sigma(z^{(l)}), \qquad \hat{p} = \text{softmax}(z^{(L+1)}) \tag{8.6}$$

Stage 8 — Compute the loss

$$J(\theta) = -\frac{1}{B}\sum_{i\in\mathcal{B}}\sum_{k=1}^{K} y_k^{(i)} \log \hat{p}_k^{(i)} + \frac{\lambda}{2}\sum_{l}\|W^{(l)}\|^2 \tag{8.7}$$

Decision to fix: the loss, which follows from the likelihood (cross-entropy for classification, MSE for regression), plus the reduction (mean over the batch).

Stage 9 — Back-propagation

Apply the chain rule backwards from the loss to the parameters. The starting point is the clean output-layer error

$$\delta^{(L+1)} = \frac{\partial J}{\partial z^{(L+1)}} = \hat{p} - y \tag{8.8}$$

and then recursively, for $l = L, \ldots, 1$:

$$\delta^{(l)} = (W^{(l+1)\top}\delta^{(l+1)}) \odot \sigma'(z^{(l)}), \qquad \frac{\partial J}{\partial W^{(l)}} = \delta^{(l)}a^{(l-1)\top}, \qquad \frac{\partial J}{\partial b^{(l)}} = \delta^{(l)} \tag{8.9}$$

The cost is one forward and one backward pass per batch — the same order as a single evaluation of the loss, which is what makes training feasible.

Stage 10 — Update the parameters

$$\theta \leftarrow \theta - \eta\,\hat{g}, \qquad \hat{g} = \frac{1}{B}\sum_{i \in \mathcal{B}} \nabla_\theta \ell_i, \qquad \mathrm{Var}(\hat{g}) \propto \frac{1}{B} \tag{8.10}$$

Decisions to fix: optimiser (SGD with momentum or Adam), learning rate η and its schedule, batch size B, number of epochs, and gradient clipping threshold (Question 3).

Stage 11 — Validate and tune

After each epoch, evaluate on $\mathcal{D}_{val}$ in *eval* mode. Track both losses, keep the best checkpoint, and stop when validation loss has not improved for a patience of p epochs. Never tune on $\mathcal{D}_{test}$.

$$\theta^{*} = \arg\min_{\theta}\ \mathrm{Err}_{val}(\theta) \tag{8.11}$$

Stage 12 — Evaluate once and deploy

Evaluate the frozen best checkpoint on $\mathcal{D}_{test}$, report the metric with a confidence interval and a confusion matrix, then monitor the deployed model for distribution drift.

The decisions that must be fixed before training

Decision	Options	Fixed by
Input representation	raw / standardised / PCA-reduced	correlation analysis, validation
Output unit and loss	softmax+CE, sigmoid+BCE, identity+MSE	the target distribution (Q5)
Depth and width	$L \in \{1..4\}$, $h \in \{32..512\}$	validation performance (Q7)
Activation	ReLU / tanh / GELU	depth and stability
Initialisation	He / Xavier	activation function
Regularisation	λ, dropout, early stopping	validation (Q1)
Optimiser and η	SGD+momentum / Adam; η log-grid	validation (Unit I, Q5)
Batch size and epochs	$B \in \{32, 64, 128\}$; $E \in [10, 200]$	memory and learning curve
Data split and metric	70/15/15, stratified; accuracy or F1	task requirements

Complete algorithm

1. Split: $\mathcal{D} \to \mathcal{D}_{train} + \mathcal{D}_{val} + \mathcal{D}_{test}$; deduplicate first.
2. Fit the preprocessing on $\mathcal{D}_{train}$; transform all three splits with those statistics.
3. Initialise θ_0 with He/Xavier; fix η, B, L, h, λ, E.
4. For each epoch $e = 1, \ldots, E$: shuffle $\mathcal{D}_{train}$; for each mini-batch compute stages 7–9 and update by stage 10.
5. After each epoch compute E_{train} and E_{val}; save the weights with the best E_{val}.
6. Stop on patience or on the epoch budget; restore the best checkpoint.
7. Evaluate once on $\mathcal{D}_{test}$; report metric, confusion matrix and the learning curves.

> **WHY THE ORDER MATTERS**
> Every stage constrains the next: the target type fixes the loss, the loss fixes the output unit, the depth fixes the initialisation scheme, and the split fixes what may be measured. Changing a decision after seeing the test score invalidates the estimate — which is why stages 1–6 are called "decisions fixed before training".

> **GOLDEN LINE**
> Split first, fit preprocessing on the training split only, fix architecture/loss/optimiser, then iterate forward pass → loss → back-propagation → update, select on validation, and touch the test set exactly once.

KEYWORDS TO WRITE IN THE EXAM

End-to-end workflow · data partition · preprocessing leakage · architecture · initialisation · forward pass · back-propagation · chain rule · mini-batch · optimiser · early stopping · validation · final test evaluation

Rapid revision — the whole unit in one grid

Read this column-wise the night before the examination: the trigger phrase, the three things that must appear, and the trap that costs marks.

Question	Three things that must appear	The trap
1 Bias–variance	gap $= E_{val} - E_{train}$; the three-term decomposition; learning curves	saying "add capacity" for a model that already fits
2 Validation	parameters/hyperparameters/test separation; k-fold; preprocessing inside the fold	fitting the scaler on all the data
3 Batch GD vs SGD	same update, different estimator; $\mathrm{Var} \propto 1/B$; m/B updates per epoch	claiming SGD converges faster per iteration
4 First intervention	diagnosis from the two errors; variance remedies in cost order; learning-curve test	changing architecture for an overfitted model
5 MLE and losses	NLL principle; Gaussian → MSE; Bernoulli → BCE; output unit follows the parameter	pairing sigmoid with MSE
6 Multiclass network	ReLU hidden layers; K logits; softmax; categorical CE; one-hot targets	independent sigmoids for exclusive classes
7 Depth and width	universal approximation (width); exponential regions (depth); parameter formulas	equating capacity only with parameter count
8 End-to-end workflow	split first; forward/loss/backward/update; validate; test once; the pre-training decisions	tuning on the test set